Preface - Unity Graphics Programming Volume 2

About This Book

This book is the second volume in the “Unity Graphics Programming” series, which explains graphics programming techniques using Unity.

In this series, we cover various topics driven by the interests of the authors, providing both introductory content for beginners along with practical applications, as well as tips for intermediate and advanced users.

Source Code Repository

The source code discussed in each chapter is available on GitHub: - **Repository**: https://github.com/IndieVisualLab/UnityGraphicsProgramming2

We encourage you to follow along with the book while running the code locally.

Reading Guide

The difficulty level varies across chapters, and depending on your existing knowledge, some content may feel too simple or too challenging. We recommend selecting articles based on your knowledge level and the topics that interest you.

For Professional Developers: If you work with graphics programming professionally, we hope this book will expand your repertoire of effects and techniques.

For Students: If you’re a student interested in visual coding with experience in Processing or openFrameworks but find 3DCG intimidating, we hope Unity can serve as a gateway to understanding the expressive power of 3DCG and provide a starting point for development.

About IndieVisualLab

IndieVisualLab is a circle founded by colleagues (and former colleagues) from work. Professionally, we use Unity for content programming in exhibition works typically categorized as “media art” - utilizing Unity in ways that differ somewhat from game development.

Throughout this book, you may find knowledge scattered throughout that proves useful when applying Unity in exhibition contexts.

Recommended Environment

Some content in this book uses Compute Shaders and Geometry Shaders, so we recommend an environment that supports DirectX 11. However, some chapters are complete with CPU-side programming (C#) alone.

If sample code behavior doesn’t work correctly due to environmental differences, please report issues to the GitHub repository or adapt the code accordingly.

Feedback and Requests

If you have feedback, concerns, or requests (such as wanting explanations on specific topics), please let us know via: - **Web Form**: Available through the QR code in the original publication - **Email**: lab.indievisual@gmail.com

---

Translation note: This preface establishes the context for Volume 2 of the Unity Graphics Programming series, emphasizing the practical, hands-on approach that characterizes the entire series.

Chapter 1: Real-Time GPU-Based Voxelizer

Original author: Nakamura Translation and annotations by Claude

---

Overview

In this chapter, we develop a GPU Voxelizer - a program that performs real-time voxelization of meshes using the GPU.

The sample code for this chapter is available in the "RealTimeGPUBasedVoxelizer" folder at: https://github.com/IndieVisualL

We'll start by examining the voxelization procedure and results using a CPU implementation, then explain the GPU implementation approach, and finally introduce an effect example that leverages high-speed voxelization.

What You'll Learn

- What voxels are and their applications in games and visualization
- The SAT (Separating Axis Theorem) algorithm for triangle-box intersection testing
- How to parallelize voxelization for GPU execution
- A practical application: GPU particle systems driven by voxel data

---

What is a Voxel?

A **voxel** represents a basic unit in a 3D regular grid space. You can think of it as a pixel with one additional dimension - where a pixel is the basic unit in a 2D regular grid space. The name "voxel" comes from combining **Volume** with **Pixel**.

Voxels can represent volume, and by storing values such as density in each voxel, this data format is used for visualization and analysis in medical and scientific applications.

In gaming, **Minecraft** is a well-known example of voxel-based graphics.

While creating detailed models and stages can be labor-intensive, voxel models can be created with relatively less effort. Excellent free editors like **MagicaVoxel** are available, allowing you to create models like 3D pixel art.

Why Voxels Matter

Voxels bridge the gap between continuous 3D surfaces and discrete volumetric data: - **Medical imaging**: CT and MRI scans naturally produce voxel data - **Destruction systems**: Voxels make it easy to carve, break, and modify geometry - **Stylized graphics**: The blocky aesthetic has become iconic - **Volume rendering**: Smoke, clouds, and other volumetric effects

Converting meshes to voxels opens up these possibilities for any 3D model.

References: - Minecraft: https://minecraft.net - MagicaVoxel: http://ephtracy.github.io/

Voxelization Algorithm

Let's explain the voxelization algorithm based on the CPU implementation. The CPU implementation is written in `CPUVoxelizer.cs`.

Overview of the Voxelization Process

The general flow of voxelization is as follows:

1. Set the voxel resolution
2. Define the range for voxelization
3. Generate a 3D array to store voxel data
4. Generate voxels located on the mesh surface
5. Fill interior voxels based on the surface voxel data

CPU voxelization is executed by calling the static function in the `CPUVoxelizer` class:

```
public class CPUVoxelizer
{
    public static void Voxelize (
        Mesh mesh,
        int resolution,
        out List<Vector3> voxels,
        out float unit,
        bool surfaceOnly = false
    ) {
    ...
    }
    ...
}
```

Specify the mesh to voxelize and the resolution as arguments. The function returns the voxel array `voxels` and the size of a single voxel `unit` through reference parameters.

The following sections explain what happens inside the `Voxelize` function, following the general flow.

Step 1: Setting the Voxel Resolution

To perform voxelization, you first set the voxel resolution. The finer the resolution, the smaller the cubes that construct the model, resulting in a more detailed voxel model - but at the cost of increased computation time.

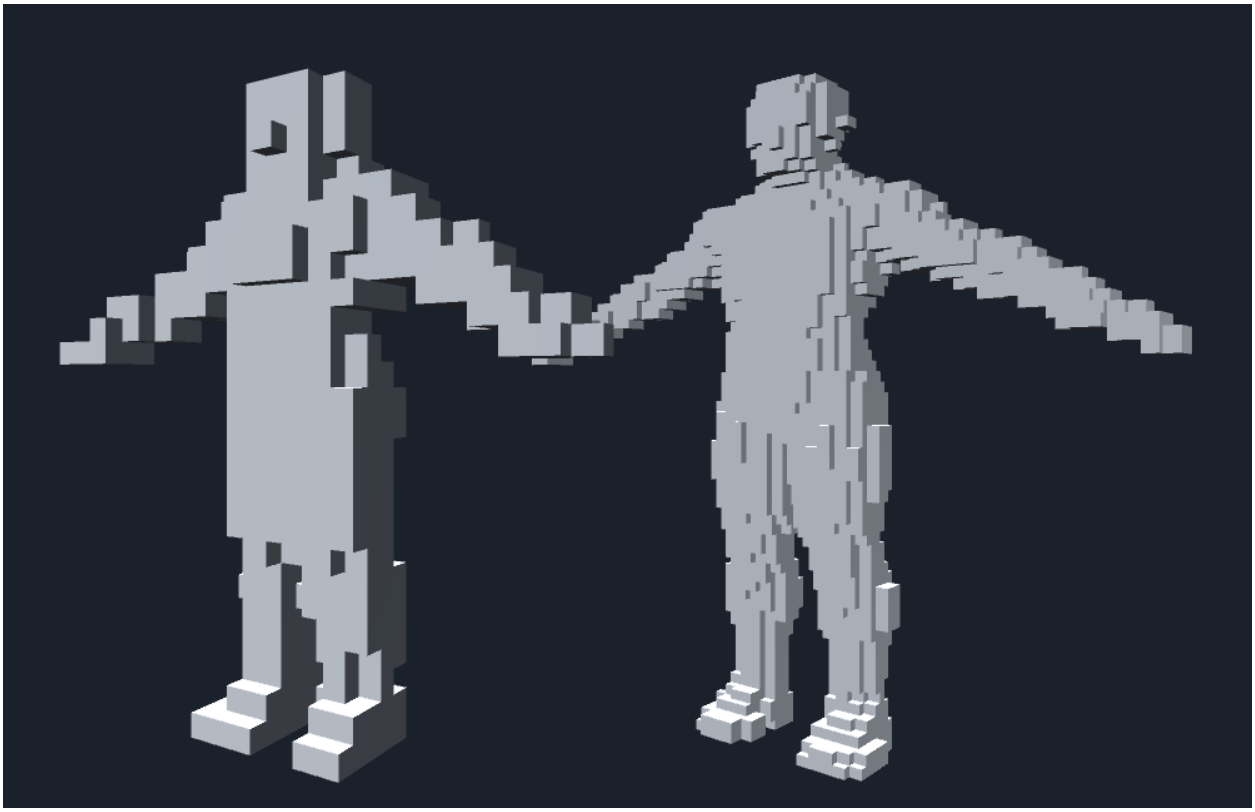


Figure: Difference in voxel resolution

Choosing Resolution

Resolution is a trade-off between quality and performance: - **Low resolution (16-32)**: Fast, chunky look, good for effects - **Medium resolution (64-128)**: Balanced, recognizable shapes - **High resolution (256+)**: Detailed but expensive, mainly for static content

For real-time applications on animated meshes, you'll typically use lower resolutions.

Step 2: Setting the Voxelization Range

Specify the range to voxelize the target mesh model. By using the mesh's **Bounding Box** (the smallest rectangular box containing all vertices) as the voxelization range, you can voxelize the entire mesh model.

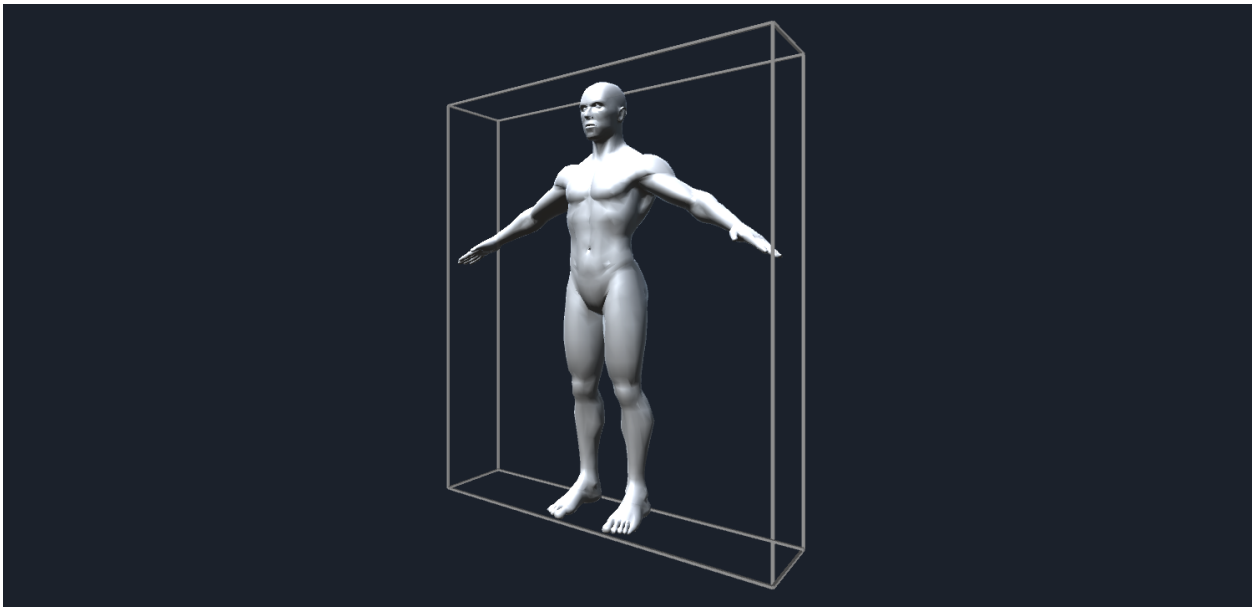


Figure: Mesh Bounding Box

One important consideration: if you use the mesh's bounding box directly as the voxelization range, problems occur when voxelizing meshes with faces that exactly align with the bounding box (like a Cube mesh).

As explained later, voxelization performs intersection tests between triangles and voxels. When a triangle face exactly aligns with a voxel face, intersection detection may not work correctly.

Therefore, we extend the mesh's bounding box by **half the voxel unit length** and use that as the voxelization range:

```
mesh.RecalculateBounds();
var bounds = mesh.bounds;

// Calculate the unit length of a single voxel from the specified resolution
float maxLength = Mathf.Max(
    bounds.size.x,
    Mathf.Max(bounds.size.y, bounds.size.z)
);
unit = maxLength / resolution;

// Half the unit length
var hunit = unit * 0.5f;

// Use the range extended by "half the unit length" as the voxelization range

// Minimum value of the voxelization bounds
var start = bounds.min - new Vector3(hunit, hunit, hunit);

// Maximum value of the voxelization bounds
var end = bounds.max + new Vector3(hunit, hunit, hunit);

// Size of the voxelization bounds
var size = end - start;
```

Step 3: Generating the 3D Array for Voxel Data

The sample code provides a `Voxel_t` structure to represent voxels:

```
[StructLayout(LayoutKind.Sequential)]
public struct Voxel_t {
    public Vector3 position;   // Voxel position
    public uint fill;          // Flag indicating whether this voxel should be filled
    public uint front;         // Flag indicating if the intersecting triangle faces forward
    ...
}
```

We generate a 3D array of `Voxel_t` and store voxel data in it:

```
// Determine the 3D voxel data size based on voxel unit length and voxelization range
var width = Mathf.CeilToInt(size.x / unit);
var height = Mathf.CeilToInt(size.y / unit);
var depth = Mathf.CeilToInt(size.z / unit);
var volume = new Voxel_t[width, height, depth];
```

Since subsequent processing references each voxel's position and size, we pre-generate an AABB array matching the 3D voxel data:

```
var boxes = new Bounds[width, height, depth];
var voxelUnitSize = Vector3.one * unit;
for(int x = 0; x < width; x++)
{
    for(int y = 0; y < height; y++)
    {
        for(int z = 0; z < depth; z++)
        {
            var p = new Vector3(x, y, z) * unit + start;
            var aabb = new Bounds(p, voxelUnitSize);
            boxes[x, y, z] = aabb;
        }
    }
}
```

What is an AABB?

AABB (Axis-Aligned Bounding Box) refers to a rectangular bounding volume with edges parallel to the X, Y, and Z axes of 3D space.

AABBs are commonly used for collision detection - for simplified collision checks between two meshes, or between a mesh and a ray.

Performing exact collision detection against a mesh requires testing all triangles, but testing against just the containing AABB is much faster.

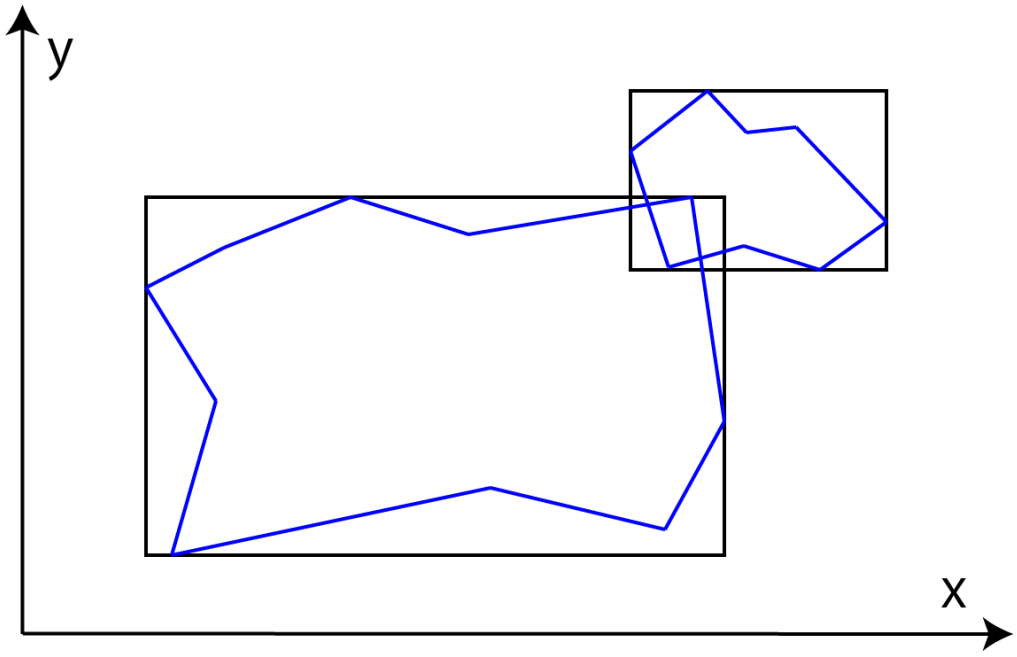


Figure: AABB collision detection between two polygon objects

Step 4: Generating Surface Voxels

As shown in the figure below, we first generate voxels located on the mesh surface:

Figure: First generate surface voxels, then use them to fill the interior

To find voxels on the mesh surface, we need to perform intersection tests between each triangle composing the mesh and the voxels.

Triangle-Voxel Intersection Testing (SAT Algorithm)

For triangle-voxel intersection testing, we use the **SAT (Separating Axis Theorem)**. The SAT-based intersection algorithm can be used generally for intersection testing between any convex shapes, not just triangles and voxels.

> **Understanding the Separating Axis Theorem**
>
> The SAT states: If you can find a line (in 2D) or plane (in 3D) that completely separates two convex objects, with all of object A on one side and all of object B on the other, then the objects do **not** intersect.
>
> This separating line/plane is always perpendicular to what's called a **separating axis**.
>
> The key insight: for convex shapes, you only need to test a **finite set of potential separating axes**. If none of them separate the objects, they must be intersecting!

Using SAT, if we can find an axis (separating axis) where the projections of two convex shapes don't overlap, then a separating line exists and we can conclude the shapes don't intersect. Conversely, if no separating axis is found, the two convex shapes are intersecting.

(Note: For concave shapes, even if no separating axis is found, the shapes might not be intersecting.)

When a convex shape is projected onto an axis, its shadow appears on that axis as a line segment, representable as a range interval [min, max].

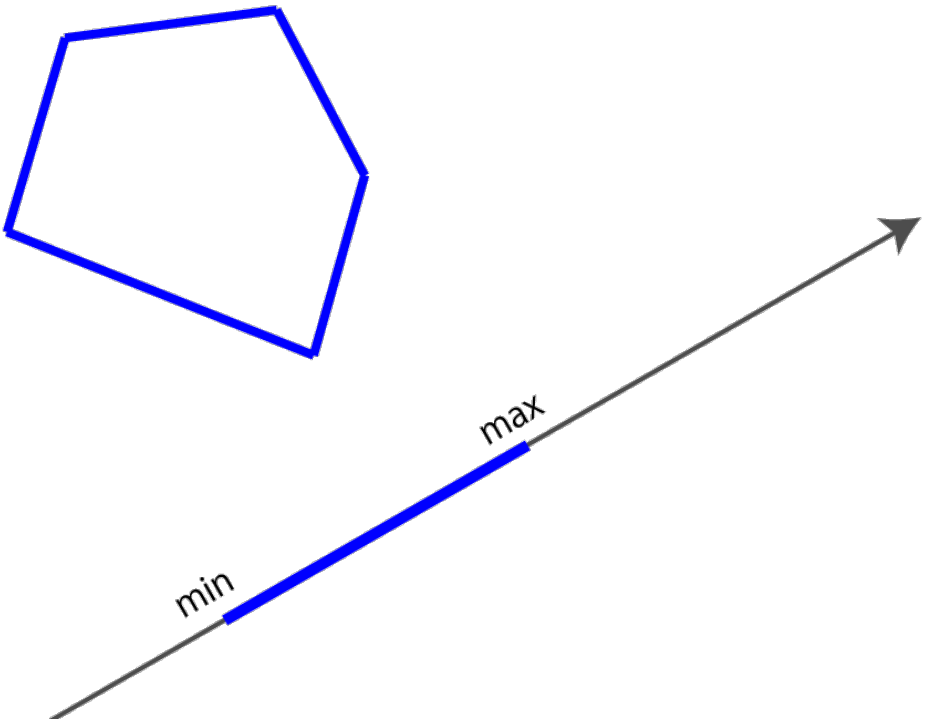


Figure: Projecting a convex shape onto an axis, showing the projected range (min, max)

As shown below, when a separating line exists between two convex shapes, the projection intervals onto the perpendicular separating axis don't overlap:

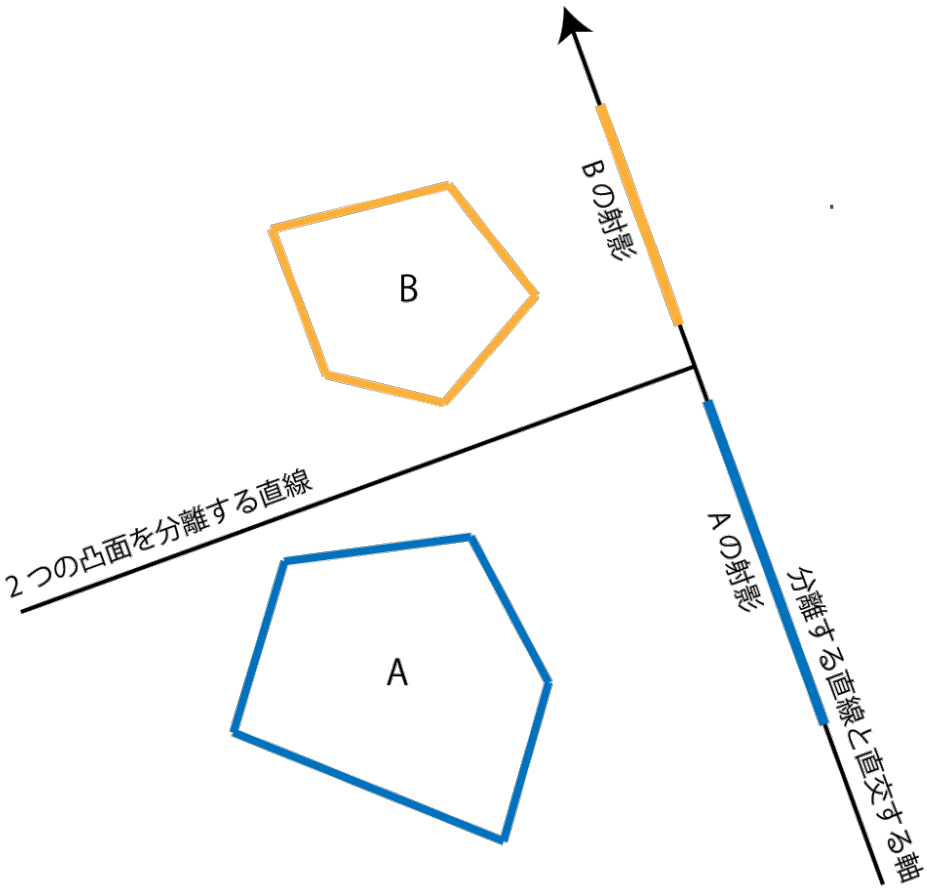


Figure: When a separating line exists, projections onto the perpendicular axis don't overlap

However, for the same two convex shapes, projections onto other (non-separating) axes may overlap:

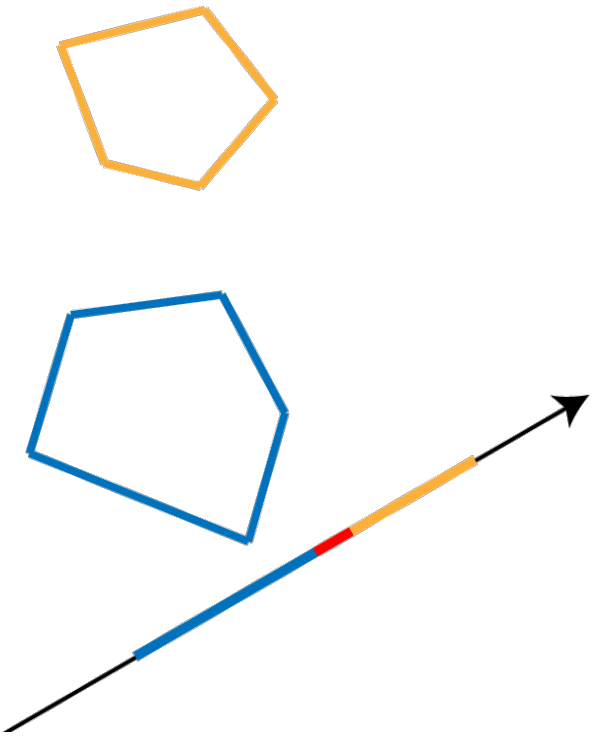


Figure: Projections onto non-separating axes may overlap

For certain shapes, the potential separating axes are well-defined. To test intersection between two such shapes A and B, project both shapes onto each potential separating axis and check if the projection intervals [Amin, Amax] and [Bmin, Bmax] overlap.

Mathematically: if `Amax < Bmin` or `Bmax < Amin`, the two intervals don't overlap.

> **The 13 Axes for Triangle-AABB Testing**
>
> For convex shapes, potential separating axes are: - Cross products of edges from shape 1 and shape 2 - Face normals of shape 1 - Face normals of shape 2
>
> For a triangle and an AABB, this gives us: - **9 axes**: Cross products of the triangle's 3 edges with the AABB's 3 edge directions - **3 axes**: The AABB's face normals (X, Y, Z axes) - **1 axis**: The triangle's face normal
>
> Total: **13 axes** to test

Implementation: Triangle-AABB Intersection

Testing every triangle against every voxel would be wasteful. Instead, we compute the AABB of each triangle and only test against voxels that might intersect:

```
// Calculate triangle's AABB
var min = tri.bounds.min - start;
var max = tri.bounds.max - start;
int iminX = Mathf.RoundToInt(min.x / unit);
int iminY = Mathf.RoundToInt(min.y / unit);
int iminZ = Mathf.RoundToInt(min.z / unit);
int imaxX = Mathf.RoundToInt(max.x / unit);
int imaxY = Mathf.RoundToInt(max.y / unit);
int imaxZ = Mathf.RoundToInt(max.z / unit);
iminX = Mathf.Clamp(iminX, 0, width - 1);
```

```
iminY = Mathf.Clamp(iminY, 0, height - 1);
iminZ = Mathf.Clamp(iminZ, 0, depth - 1);
imaxX = Mathf.Clamp(imaxX, 0, width - 1);
imaxY = Mathf.Clamp(imaxY, 0, height - 1);
imaxZ = Mathf.Clamp(imaxZ, 0, depth - 1);

// Test intersection within the triangle's AABB
for(int x = iminX; x <= imaxX; x++) {
    for(int y = iminY; y <= imaxY; y++) {
        for(int z = iminZ; z <= imaxZ; z++) {
            if(Intersects(tri, boxes[x, y, z])) {
                ...
            }
        }
    }
}
```

The `Intersects(Triangle, Bounds)` function performs the actual intersection test:

```
public static bool Intersects(Triangle tri, Bounds aabb)
{
    ...
}
```

This function tests the 13 axes. Several optimizations are applied: - The AABB's normals are known (they're simply the X, Y, Z axes) - We translate coordinates so the AABB center is at the origin (0, 0, 0)

```
// Get AABB center and half-extents
Vector3 center = aabb.center, extents = aabb.max - center;

// Translate triangle vertices so AABB center is at origin
Vector3 v0 = tri.a - center,
        v1 = tri.b - center,
        v2 = tri.c - center;

// Get vectors representing the triangle's three edges
Vector3 f0 = v1 - v0,
        f1 = v2 - v1,
        f2 = v0 - v2;
```

Since the AABB edges are parallel to the X, Y, Z axes, we can compute the 9 cross product axes without expensive calculations:

```
// Since AABB edges have directions x(1,0,0), y(0,1,0), z(0,0,1),
// we can obtain the 9 cross products directly
Vector3
    a00 = new Vector3(0, -f0.z, f0.y), // X-axis cross f0
    a01 = new Vector3(0, -f1.z, f1.y), // X cross f1
    a02 = new Vector3(0, -f2.z, f2.y), // X cross f2
    a10 = new Vector3(f0.z, 0, -f0.x), // Y cross f0
    a11 = new Vector3(f1.z, 0, -f1.x), // Y cross f1
    a12 = new Vector3(f2.z, 0, -f2.x), // Y cross f2
    a20 = new Vector3(-f0.y, f0.x, 0), // Z cross f0
    a21 = new Vector3(-f1.y, f1.x, 0), // Z cross f1
    a22 = new Vector3(-f2.y, f2.x, 0); // Z cross f2

// Test all 9 axes (if any don't intersect, return false)
```

```
if (
    !Intersects(v0, v1, v2, extents, a00) ||
    !Intersects(v0, v1, v2, extents, a01) ||
    !Intersects(v0, v1, v2, extents, a02) ||
    !Intersects(v0, v1, v2, extents, a10) ||
    !Intersects(v0, v1, v2, extents, a11) ||
    !Intersects(v0, v1, v2, extents, a12) ||
    !Intersects(v0, v1, v2, extents, a20) ||
    !Intersects(v0, v1, v2, extents, a21) ||
    !Intersects(v0, v1, v2, extents, a22)
)
{
    return false;
}
```

The projection and overlap test for each axis:

```
protected static bool Intersects(
    Vector3 v0,
    Vector3 v1,
    Vector3 v2,
    Vector3 extents,
    Vector3 axis
)
{
    ...
}
```

The AABB Projection Optimization

By centering the AABB at the origin, we get a powerful optimization. Instead of projecting all 8 AABB vertices onto the axis, we only need to project the extents (half-dimensions).

The projected value `r` represents the interval [-r, r] on the axis. This means one projection calculation covers the entire AABB!

```
// Project triangle vertices onto the axis
float p0 = Vector3.Dot(v0, axis);
float p1 = Vector3.Dot(v1, axis);
float p2 = Vector3.Dot(v2, axis);

// Project the AABB's maximum extent vertex onto the axis to get r
// The AABB interval is [-r, r], so we don't need to project all vertices
float r =
    extents.x * Mathf.Abs(axis.x) +
    extents.y * Mathf.Abs(axis.y) +
    extents.z * Mathf.Abs(axis.z);

// Triangle projection interval
float minP = Mathf.Min(p0, p1, p2);
float maxP = Mathf.Max(p0, p1, p2);

// Check if triangle and AABB intervals overlap
return !((maxP < -r) || (r < minP));
```

After testing the 9 cross-product axes, we test the AABB's 3 face normals. Since the AABB normals are parallel to the X, Y, Z axes and we've translated coordinates to center the AABB at the origin, we simply

compare triangle vertex coordinates against extents:

```
// X-axis
if (
    Mathf.Max(v0.x, v1.x, v2.x) < -extents.x ||
    Mathf.Min(v0.x, v1.x, v2.x) > extents.x
)
{
    return false;
}

// Y-axis
if (
    Mathf.Max(v0.y, v1.y, v2.y) < -extents.y ||
    Mathf.Min(v0.y, v1.y, v2.y) > extents.y
)
{
    return false;
}

// Z-axis
if (
    Mathf.Max(v0.z, v1.z, v2.z) < -extents.z ||
    Mathf.Min(v0.z, v1.z, v2.z) > extents.z
)
{
    return false;
}
```

Finally, we test the triangle's normal by checking plane-AABB intersection:

```
var normal = Vector3.Cross(f1, f0).normalized;
var pl = new Plane(normal, Vector3.Dot(normal, tri.a));
return Intersects(pl, aabb);
```

The plane-AABB intersection test:

```
public static bool Intersects(Plane pl, Bounds aabb)
{
    Vector3 center = aabb.center;
    var extents = aabb.max - center;

    // Project extents onto the plane normal
    var r =
        extents.x * Mathf.Abs(pl.normal.x) +
        extents.y * Mathf.Abs(pl.normal.y) +
        extents.z * Mathf.Abs(pl.normal.z);

    // Calculate distance from plane to AABB center
    var s = Vector3.Dot(pl.normal, center) - pl.distance;

    // Check if s is within [-r, r]
    return Mathf.Abs(s) <= r;
}
```

Writing Intersecting Voxels to the Array

When we find a triangle intersecting a voxel, we set the voxel's `fill` flag and the `front` flag indicating whether the triangle faces forward or backward from a specified direction:

```
if(Intersects(tri, boxes[x, y, z])) {
    // Get the voxel at position (x, y, z)
    var voxel = volume[x, y, z];

    // Set the voxel position
    voxel.position = boxes[x, y, z].center;

    if(voxel.fill & 1 == 0) {
        // If voxel isn't filled yet, set the front flag
        voxel.front = front;
    } else {
        // If voxel is already filled by another triangle,
        // prioritize back-facing
        voxel.front = voxel.front & front;
    }

    // Set the fill flag
    voxel.fill = 1;
    volume[x, y, z] = voxel;
}
```

When a voxel intersects both front-facing and back-facing triangles, we prioritize the back-facing flag.

The `front` flag is needed for the "fill interior" processing explained next. It indicates whether the triangle faces forward or backward from the "fill direction."

In the sample code, we fill the interior along the forward(0, 0, 1) direction, so we determine if triangles face forward from that direction:

```
public class Triangle {
    public Vector3 a, b, c;     // Triangle's three vertices
    public bool frontFacing;    // Whether triangle faces the fill direction
    public Bounds bounds;       // Triangle's AABB

    public Triangle (Vector3 a, Vector3 b, Vector3 c, Vector3 dir) {
        this.a = a;
        this.b = b;
        this.c = c;

        // Determine if triangle faces forward from the fill direction
        var normal = Vector3.Cross(b - a, c - a);
        this.frontFacing = (Vector3.Dot(normal, dir) <= 0f);

        ...
    }
}
```

Step 5: Filling Interior Voxels

Now that we have the surface voxel data, we fill the interior.

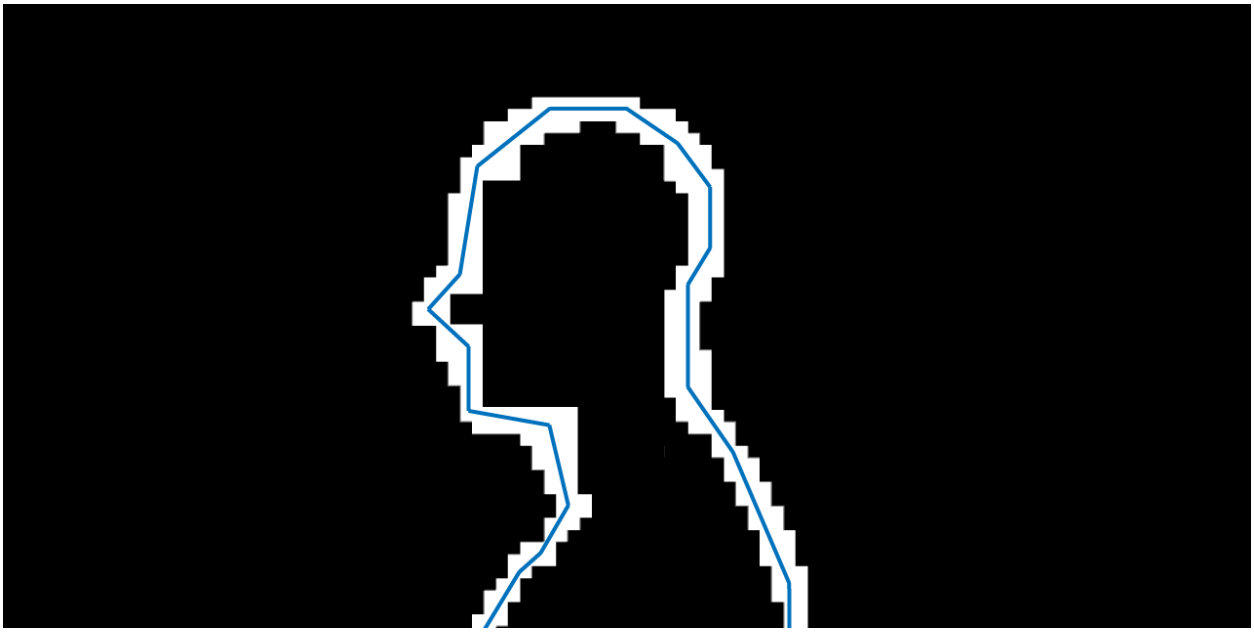


Figure: State after generating surface voxels

The Fill Algorithm

We search for voxels facing forward from the fill direction.

Empty voxels are passed through:

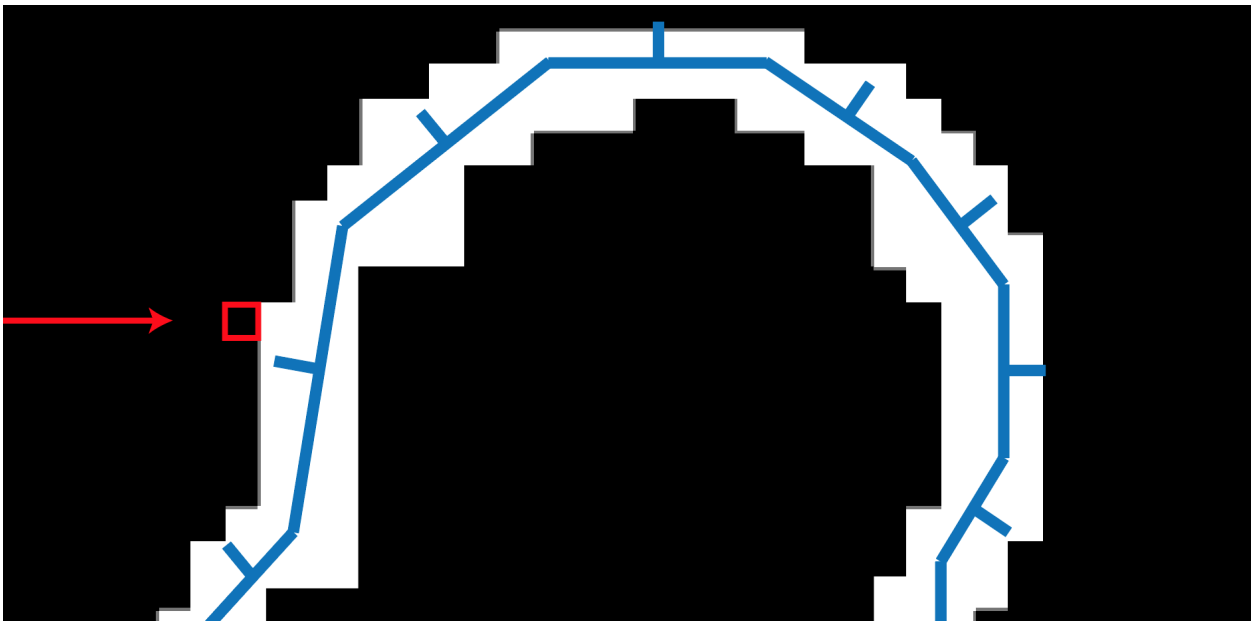


Figure: Searching for front-facing voxels, passing through empty ones (arrow = fill direction, box = current position)

When we find a front-facing voxel, we traverse through the front-facing voxels:

Figure: Found a front-facing voxel (lines from mesh are normals - opposing the fill direction means front-facing)

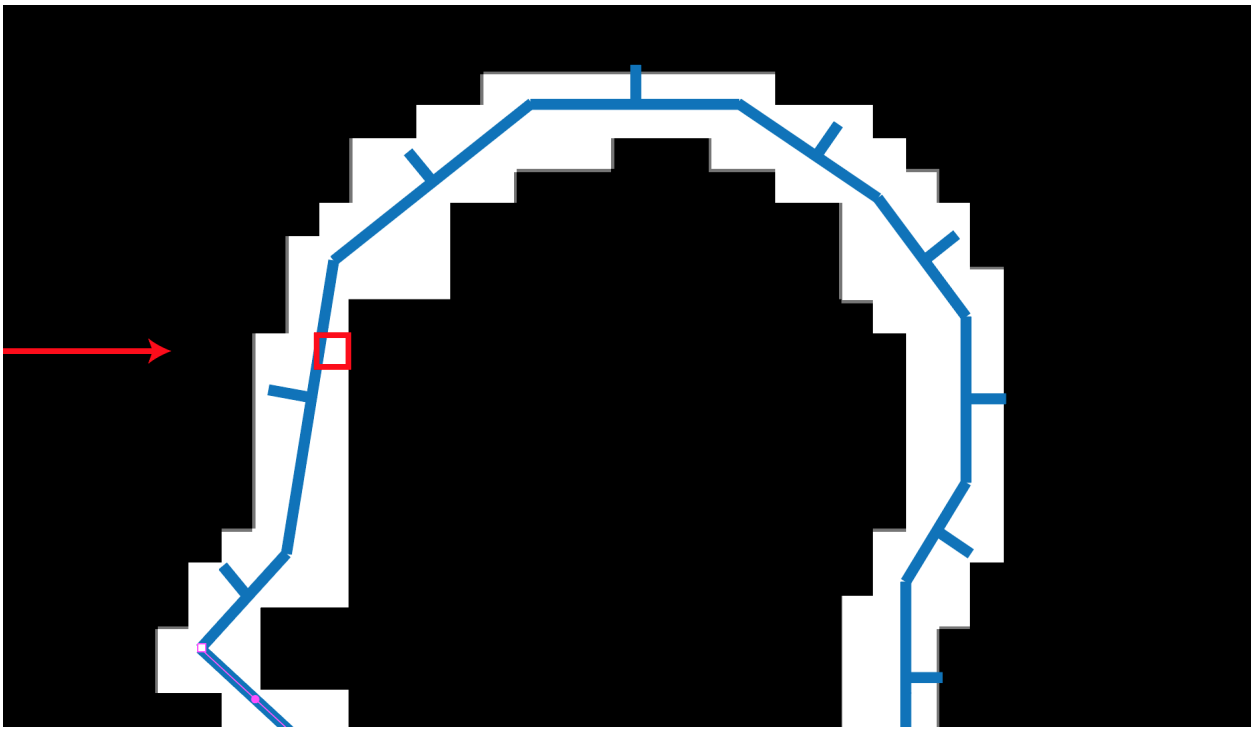


Figure: Traversing through front-facing voxels

After passing through the front-facing voxels, we reach the mesh interior:

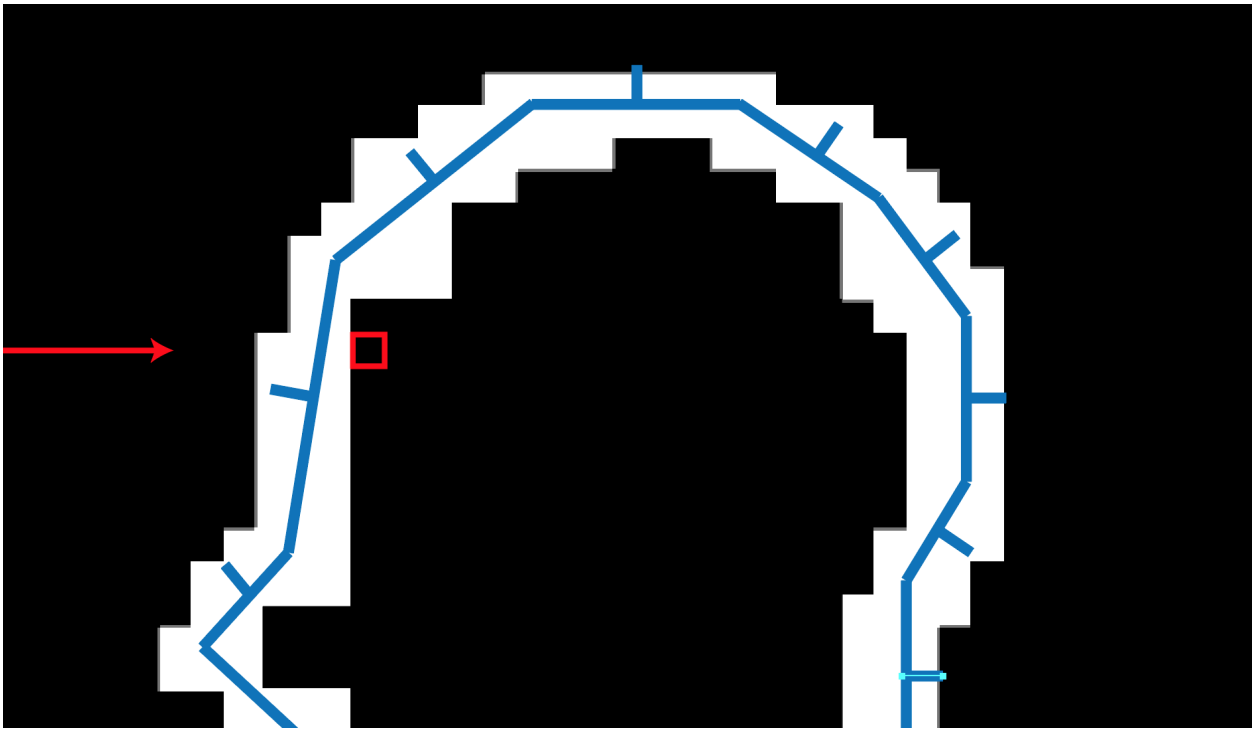


Figure: Passed through front-facing voxels, reached mesh interior

We proceed through the interior, filling each voxel we reach:

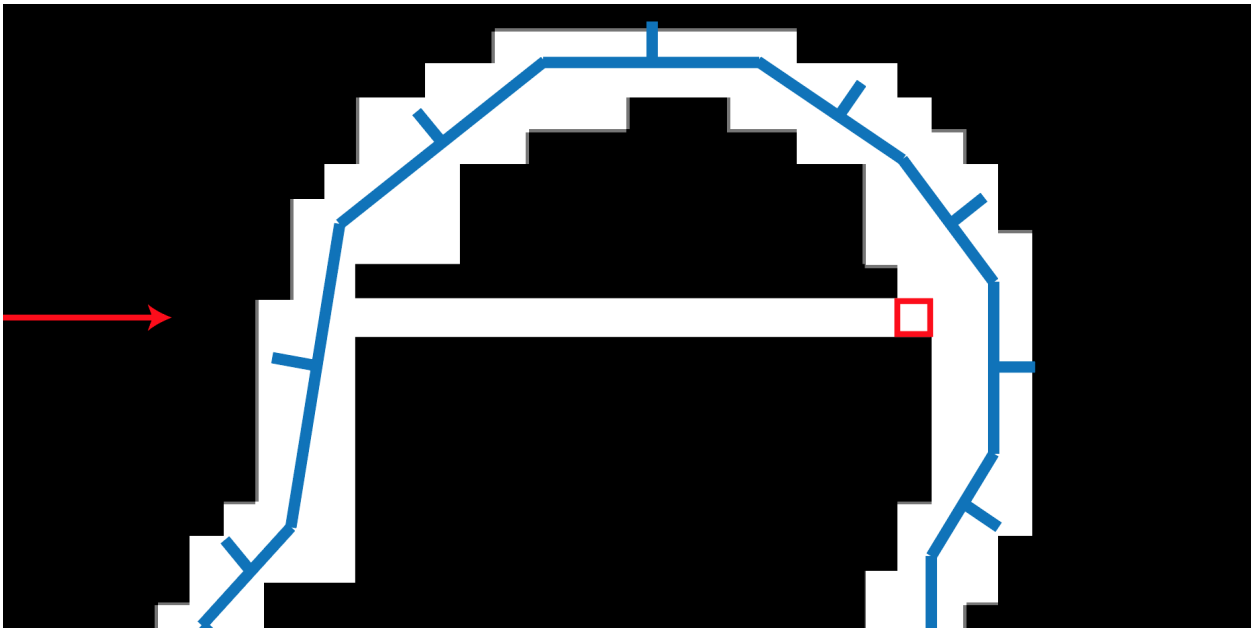


Figure: Filling voxels as we traverse the interior

When we reach a back-facing voxel, we've filled the entire interior. We traverse through the back-facing voxels and exit the mesh, then resume searching for front-facing voxels:

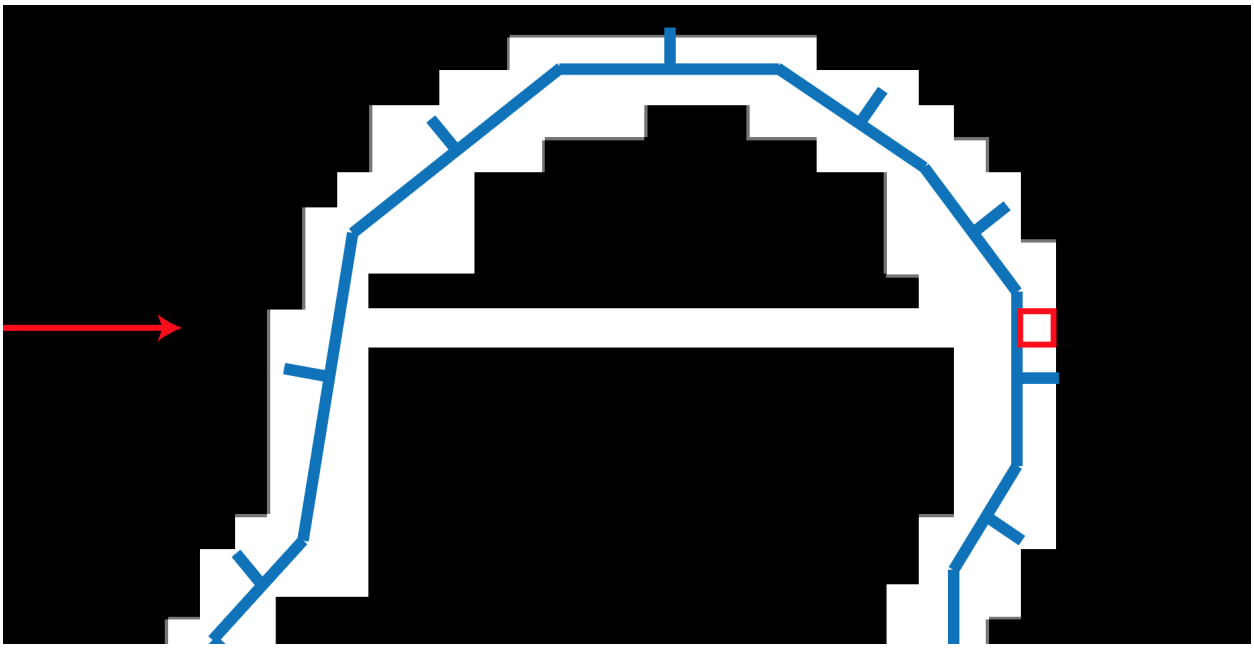


Figure: Traversing back-facing voxels and exiting the mesh

> **Understanding the Fill Logic**
>
> This is essentially a **ray marching** approach along the Z-axis: 1. **Outside mesh**: Empty voxels, keep searching 2. **Enter mesh**: Hit front-facing surface voxels 3. **Inside mesh**: Fill all voxels 4. **Exit mesh**: Hit back-facing surface voxels 5. **Outside mesh again**: Back to step 1
>
> By prioritizing back-facing flags on surface voxels, we ensure proper handling of thin geometry where front and back faces share the same voxel.

Fill Implementation

Since we fill along the forward(0, 0, 1) direction, we iterate along the Z direction in the 3D voxel array:

```
// Fill the mesh interior
for(int x = 0; x < width; x++)
{
    for(int y = 0; y < height; y++)
    {
        // Fill from front to back along Z
        for(int z = 0; z < depth; z++)
        {
            ...
        }
    }
}
```

Using the `front` flag stored in each voxel, we implement the fill process:

```
...
// Fill from front to back along Z
for(int z = 0; z < depth; z++)
{
    // Skip if (x, y, z) is empty
    if (volume[x, y, z].IsEmpty()) continue;
```

```
    // Traverse front-facing voxels
    int ifront = z;
    for(; ifront < depth && volume[x, y, ifront].IsFrontFace(); ifront++) {}

    // Done if we reached the end
    if(ifront >= depth) break;

    // Search for back-facing voxels
    int iback = ifront;

    // Traverse through the interior
    for (; iback < depth && volume[x, y, iback].IsEmpty(); iback++) {}

    // Done if we reached the end
    if (iback >= depth) break;

    // Check if (x, y, iback) is back-facing
    if(volume[x, y, iback].IsBackFace()) {
        // Traverse back-facing voxels
        for (; iback < depth && volume[x, y, iback].IsBackFace(); iback++) {}
    }

    // Fill voxels from (x, y, ifront) to (x, y, iback)
    for(int z2 = ifront; z2 < iback; z2++)
    {
        var p = boxes[x, y, z2].center;
        var voxel = volume[x, y, z2];
        voxel.position = p;
        voxel.fill = 1;
        volume[x, y, z2] = voxel;
    }

    // Advance the loop to the processed position
    z = iback;
}
```

At this point, we have voxel data with the mesh interior filled.

The 3D voxel data contains empty voxels, so `CPUVoxelizer.Voxelize` returns only the voxels representing the surface and filled interior:

```
// Get non-empty voxels
voxels = new List<Voxel_t>();
for(int x = 0; x < width; x++) {
    for(int y = 0; y < height; y++) {
        for(int z = 0; z < depth; z++) {
            if(!volume[x, y, z].IsEmpty())
            {
                voxels.Add(volume[x, y, z]);
            }
        }
    }
}
```

The `CPUVoxelizerTest.cs` script builds and visualizes a mesh using the voxel data from `CPUVoxelizer`.

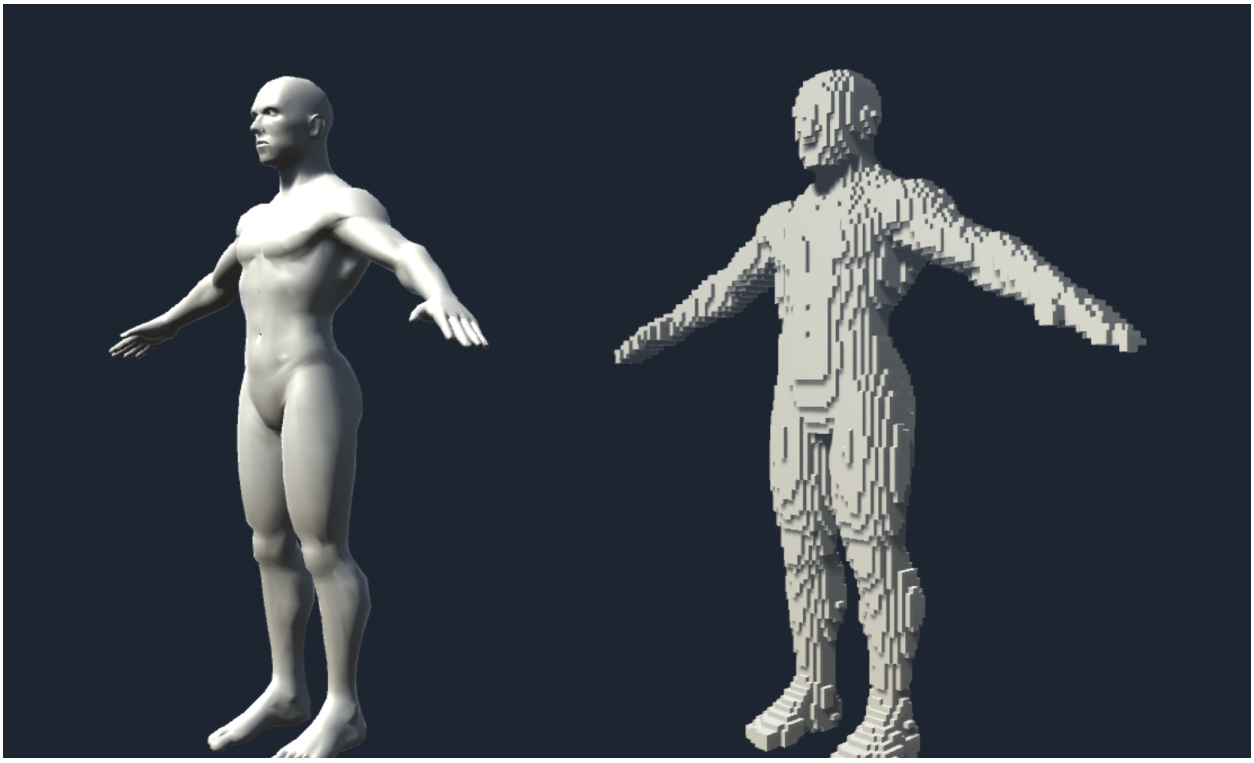


Figure: Voxel data from CPUVoxelizer.Voxelize visualized as a mesh (CPUVoxelizerTest.scene)

Voxel Mesh Representation

The `VoxelMesh` class contains processing to build a mesh from the `Voxel_t[]` array and voxel unit size information.

The `CPUVoxelizerTest.cs` from the previous section uses this class to generate voxel meshes:

```
public class VoxelMesh {

    public static Mesh Build (Voxel_t[] voxels, float size)
    {
        var hsize = size * 0.5f;
        var forward = Vector3.forward * hsize;
        var back = -forward;
        var up = Vector3.up * hsize;
        var down = -up;
        var right = Vector3.right * hsize;
        var left = -right;

        var vertices = new List<Vector3>();
        var normals = new List<Vector3>();
        var triangles = new List<int>();

        for(int i = 0, n = voxels.Length; i < n; i++)
        {
            if(voxel[i].fill == 0) continue;
```

```
            var p = voxels[i].position;

            // 8 corner vertices forming a cube for one voxel
            var corners = new Vector3[8] {
                p + forward + left + up,
                p + back + left + up,
                p + back + right + up,
                p + forward + right + up,

                p + forward + left + down,
                p + back + left + down,
                p + back + right + down,
                p + forward + right + down,
            };

            // Build the 6 faces of the cube
            // (up, down, right, left, forward, back faces)
            ...
        }

        var mesh = new Mesh();
        mesh.SetVertices(vertices);

        // Use 32-bit index format if vertex count exceeds 16-bit support
        mesh.indexFormat =
            (vertices.Count <= 65535)
            ? IndexFormat.UInt16 : IndexFormat.UInt32;
        mesh.SetNormals(normals);
        mesh.SetIndices(triangles.ToArray(), MeshTopology.Triangles, 0);
        mesh.RecalculateBounds();
        return mesh;
    }
}
```

GPU Implementation

Now we'll explain how to perform the voxelization implemented in `CPUVoxelizer` much faster using the GPU.

The voxelization algorithm from `CPUVoxelizer` can be parallelized across each coordinate in a grid on the XY plane, divided by the voxel unit length.

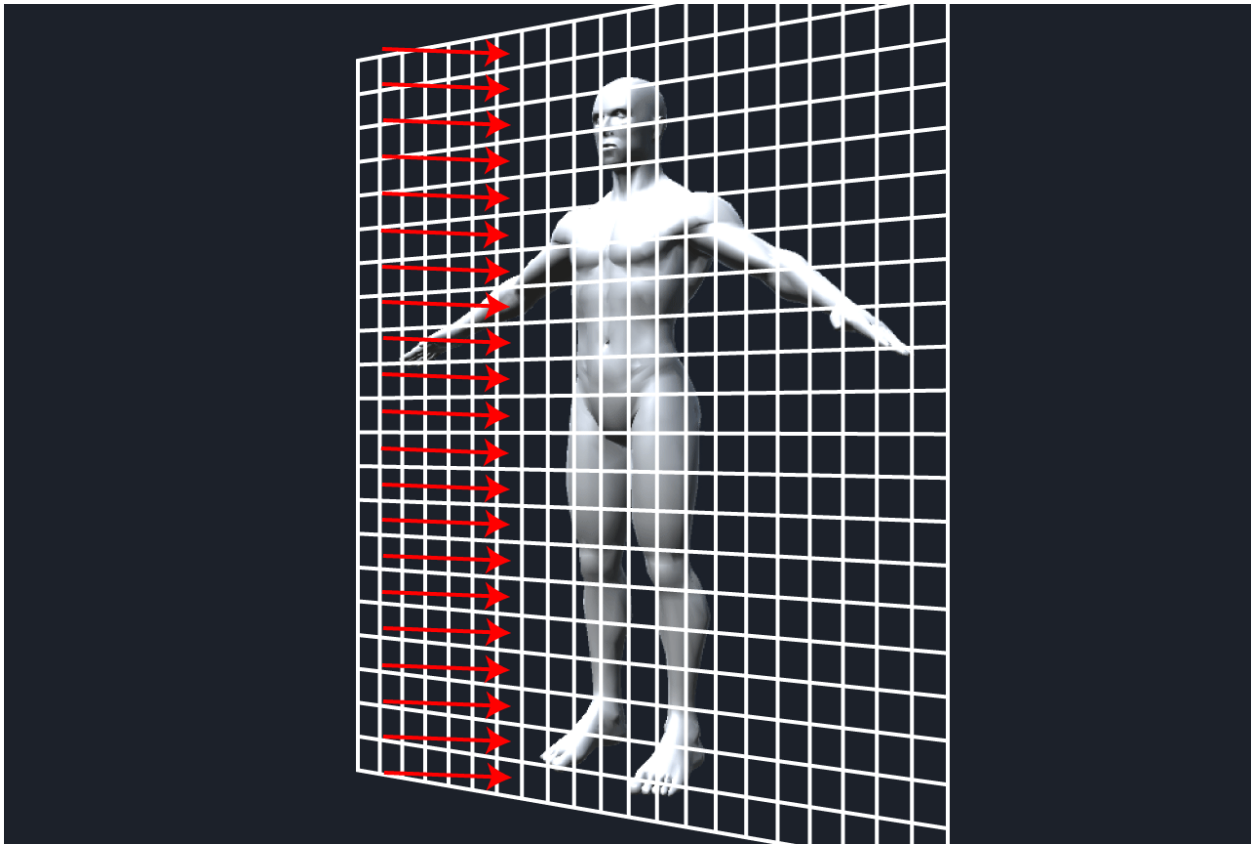


Figure: Grid on XY plane divided by voxel unit length - voxelization can be parallelized per grid cell for GPU implementation

By assigning each parallelizable operation to a GPU thread, we can benefit from the GPU's high-speed parallel computation.

The GPU voxelization implementation is in `GPUVoxelizer.cs` and `Voxelizer.compute`.

> **Prerequisites**
>
> This section uses **Compute Shaders** for GPGPU programming. If you're unfamiliar with Compute Shaders, please refer to Unity Graphics Programming Volume 1, "Introduction to Compute Shaders" for the basics.

GPU voxelization is executed by calling the static function:

```
public class GPUVoxelizer
{
    public static GPUVoxelData Voxelize (
        ComputeShader voxelizer,
        Mesh mesh,
        int resolution
    ) {
    ...
    }
}
```

Specify `Voxelizer.compute`, the mesh to voxelize, and the resolution. The function returns `GPUVoxelData` containing the voxel data.

GPU Voxelization Data Setup

We perform the same setup as the CPU implementation (steps 1-3):

```
public static GPUVoxelData Voxelize (
    ComputeShader voxelizer,
    Mesh mesh,
    int resolution
) {
    // Same processing as CPUVoxelizer.Voxelize -------
    mesh.RecalculateBounds();
    var bounds = mesh.bounds;

    float maxLength = Mathf.Max(
        bounds.size.x,
        Mathf.Max(bounds.size.y, bounds.size.z)
    );
    var unit = maxLength / resolution;

    var hunit = unit * 0.5f;

    var start = bounds.min - new Vector3(hunit, hunit, hunit);
    var end = bounds.max + new Vector3(hunit, hunit, hunit);
    var size = end - start;

    int width = Mathf.CeilToInt(size.x / unit);
    int height = Mathf.CeilToInt(size.y / unit);
    int depth = Mathf.CeilToInt(size.z / unit);
    // ------- End same as CPUVoxelizer.Voxelize
    ...
}
```

The `Voxel_t` array is defined as a `ComputeBuffer` for GPU access. Note that unlike the CPU's 3D array, we define it as a **1D array**.

> **Why a 1D Array?**
>
> GPUs don't handle multidimensional arrays well. Instead, we use a 1D array and calculate the index from 3D coordinates (x, y, z) within the Compute Shader, effectively treating the 1D array as a 3D array.
>
> The index formula is typically: `index = x + y * width + z * width * height`

```
// Create ComputeBuffer for Voxel_t array
var voxelBuffer = new ComputeBuffer(
    width * height * depth,
    Marshal.SizeOf(typeof(Voxel_t))
);
var voxels = new Voxel_t[voxelBuffer.count];
voxelBuffer.SetData(voxels); // Initialize
```

Transfer the setup data to the GPU:

```
// Transfer voxel data to GPU
voxelizer.SetVector("_Start", start);
voxelizer.SetVector("_End", end);
voxelizer.SetVector("_Size", size);
```

```
voxelizer.SetFloat("_Unit", unit);
voxelizer.SetFloat("_InvUnit", 1f / unit);
voxelizer.SetFloat("_HalfUnit", hunit);
voxelizer.SetInt("_Width", width);
voxelizer.SetInt("_Height", height);
voxelizer.SetInt("_Depth", depth);
```

Create `ComputeBuffers` for the mesh data (needed for triangle-voxel intersection testing):

```
// Create ComputeBuffer for mesh vertex array
var vertices = mesh.vertices;
var vertBuffer = new ComputeBuffer(
    vertices.Length,
    Marshal.SizeOf(typeof(Vector3))
);
vertBuffer.SetData(vertices);

// Create ComputeBuffer for mesh triangle array
var triangles = mesh.triangles;
var triBuffer = new ComputeBuffer(
    triangles.Length,
    Marshal.SizeOf(typeof(int))
);
triBuffer.SetData(triangles);
```

GPU Surface Voxel Generation

When generating surface voxels on the GPU, we first process front-facing triangles, then back-facing triangles.

> **Why Process Front and Back Separately?**
>
> With GPU parallel computation, multiple threads might write to the same voxel simultaneously, leading to **race conditions** and undefined results.
>
> Since we want to prioritize back-facing flags, we process front-facing triangles first, then back-facing. This ensures back-facing values overwrite front-facing ones, eliminating result ambiguity.

Transfer mesh data to the `SurfaceFront` kernel:

```
// Transfer mesh data to SurfaceFront kernel
var surfaceFrontKer = new Kernel(voxelizer, "SurfaceFront");
voxelizer.SetBuffer(surfaceFrontKer.Index, "_VoxelBuffer", voxelBuffer);
voxelizer.SetBuffer(surfaceFrontKer.Index, "_VertBuffer", vertBuffer);
voxelizer.SetBuffer(surfaceFrontKer.Index, "_TriBuffer", triBuffer);

// Set triangle count
var triangleCount = triBuffer.count / 3; // (vertex indices / 3) = triangle count
voxelizer.SetInt("_TriangleCount", triangleCount);
```

This process runs in parallel per triangle. Set thread groups to `(triangleCount / threads + 1, 1, 1)`:

```
// Build voxels intersecting front-facing triangles
voxelizer.Dispatch(
    surfaceFrontKer.Index,
    triangleCount / (int)surfaceFrontKer.ThreadX + 1,
```

```
    (int)surfaceFrontKer.ThreadY,
    (int)surfaceFrontKer.ThreadZ
);
```

The `SurfaceFront` kernel processes only front-facing triangles, returning early for back-facing ones:

```
[numthreads(8, 1, 1)]
void SurfaceFront (uint3 id : SV_DispatchThreadID)
{
    // Return if exceeding triangle count
    int idx = (int)id.x;
    if(idx >= _TriangleCount) return;

    // Get triangle vertex positions and front/back flag
    float3 va, vb, vc;
    bool front;
    get_triangle(idx, va, vb, vc, front);

    // Return if back-facing
    if (!front) return;

    // Build surface voxels
    surface(va, vb, vc, front);
}
```

The `get_triangle` function retrieves triangle vertices and front/back flag from mesh data:

```
void get_triangle(
    int idx,
    out float3 va, out float3 vb, out float3 vc,
    out bool front
)
{
    int ia = _TriBuffer[idx * 3];
    int ib = _TriBuffer[idx * 3 + 1];
    int ic = _TriBuffer[idx * 3 + 2];

    va = _VertBuffer[ia];
    vb = _VertBuffer[ib];
    vc = _VertBuffer[ic];

    // Determine if triangle is front or back-facing from forward(0,0,1)
    float3 normal = cross((vb - va), (vc - vb));
    front = dot(normal, float3(0, 0, 1)) < 0;
}
```

The `surface` function performs intersection testing and writes to voxel data - nearly identical to the CPU implementation, with the addition of 1D index calculation:

```
void surface (float3 va, float3 vb, float3 vc, bool front)
{
    // Calculate triangle AABB
    float3 tbmin = min(min(va, vb), vc);
    float3 tbmax = max(max(va, vb), vc);

    float3 bmin = tbmin - _Start;
    float3 bmax = tbmax - _Start;
```

```
    int iminX = round(bmin.x / _Unit);
    int iminY = round(bmin.y / _Unit);
    int iminZ = round(bmin.z / _Unit);
    int imaxX = round(bmax.x / _Unit);
    int imaxY = round(bmax.y / _Unit);
    int imaxZ = round(bmax.z / _Unit);
    iminX = clamp(iminX, 0, _Width - 1);
    iminY = clamp(iminY, 0, _Height - 1);
    iminZ = clamp(iminZ, 0, _Depth - 1);
    imaxX = clamp(imaxX, 0, _Width - 1);
    imaxY = clamp(imaxY, 0, _Height - 1);
    imaxZ = clamp(imaxZ, 0, _Depth - 1);

    // Test intersection within triangle AABB
    for(int x = iminX; x <= imaxX; x++) {
        for(int y = iminY; y <= imaxY; y++) {
            for(int z = iminZ; z <= imaxZ; z++) {
                // Generate AABB for voxel at (x, y, z)
                float3 center = float3(x, y, z) * _Unit + _Start;
                AABB aabb;
                aabb.min = center - _HalfUnit;
                aabb.center = center;
                aabb.max = center + _HalfUnit;
                if(intersects_tri_aabb(va, vb, vc, aabb))
                {
                    // Get 1D index from (x, y, z)
                    uint vid = get_voxel_index(x, y, z);
                    Voxel voxel = _VoxelBuffer[vid];
                    voxel.position = get_voxel_position(x, y, z);
                    voxel.front = front;
                    voxel.fill = true;
                    _VoxelBuffer[vid] = voxel;
                }
            }
        }
    }
}
```

After processing front-facing triangles, process back-facing ones:

```
var surfaceBackKer = new Kernel(voxelizer, "SurfaceBack");
voxelizer.SetBuffer(surfaceBackKer.Index, "_VoxelBuffer", voxelBuffer);
voxelizer.SetBuffer(surfaceBackKer.Index, "_VertBuffer", vertBuffer);
voxelizer.SetBuffer(surfaceBackKer.Index, "_TriBuffer", triBuffer);
voxelizer.Dispatch(
    surfaceBackKer.Index,
    triangleCount / (int)surfaceBackKer.ThreadX + 1,
    (int)surfaceBackKer.ThreadY,
    (int)surfaceBackKer.ThreadZ
);
```

`SurfaceBack` is identical to `SurfaceFront` except it returns early for front-facing triangles. Running `SurfaceBack` after `SurfaceFront` ensures that voxels intersecting both front and back-facing triangles will have their `front` flag overwritten to back-facing:

```
[numthreads(8, 1, 1)]
```

```
void SurfaceBack (uint3 id : SV_DispatchThreadID)
{
    int idx = (int)id.x;
    if(idx >= _TriangleCount) return;

    float3 va, vb, vc;
    bool front;
    get_triangle(idx, va, vb, vc, front);

    // Return if front-facing
    if (front) return;

    surface(va, vb, vc, front);
}
```

GPU Interior Fill

The `Volume` kernel handles interior filling. It creates a thread for each coordinate on the XY grid - parallelizing the XY double loop from the CPU implementation:

```
// Transfer voxel data to Volume kernel
var volumeKer = new Kernel(voxelizer, "Volume");
voxelizer.SetBuffer(volumeKer.Index, "_VoxelBuffer", voxelBuffer);

// Fill mesh interior
voxelizer.Dispatch(
    volumeKer.Index,
    width / (int)volumeKer.ThreadX + 1,
    height / (int)volumeKer.ThreadY + 1,
    (int)volumeKer.ThreadZ
);
```

The `Volume` kernel implementation is nearly identical to the CPU version:

```
[numthreads(8, 8, 1)]
void Volume (uint3 id : SV_DispatchThreadID)
{
    int x = (int)id.x;
    int y = (int)id.y;
    if(x >= _Width) return;
    if(y >= _Height) return;

    for (int z = 0; z < _Depth; z++)
    {
        Voxel voxel = _VoxelBuffer[get_voxel_index(x, y, z)];
        // Nearly identical processing to CPUVoxelizer.Voxelize follows
        ...
    }
}
```

After obtaining the voxel data, we release the mesh data buffers and create `GPUVoxelData`:

```
// Release mesh data no longer needed
vertBuffer.Release();
triBuffer.Release();
```

```
return new GPUVoxelData(voxelBuffer, width, height, depth, unit);
```

GPU voxelization is now complete. `GPUVoxelizerTest.cs` demonstrates visualizing `GPUVoxelData`.

CPU vs GPU Performance Comparison

The test scenes execute the Voxelizer at Play time, making the speed difference less obvious, but GPU implementation achieves significant speedup.

Performance depends heavily on the execution environment, mesh polygon count, and voxelization resolution, but under these conditions:

- **Environment**: Windows 10, Core i7 CPU, 32GB RAM, GeForce GTX 980 GPU
- **Mesh**: 5,319 vertices, 9,761 triangles
- **Resolution**: 256

The GPU implementation runs **over 50 times faster** than the CPU implementation.

> **Why Such a Big Difference?**
>
> The CPU processes XY grid positions sequentially (nested loops). With resolution 256, that's 256 x 256 = 65,536 sequential iterations.
>
> The GPU processes all 65,536 positions **simultaneously** across thousands of parallel threads. Each thread handles one Z-column independently.
>
> Additionally, the triangle intersection tests are parallelized per-triangle on the GPU, whereas the CPU must test triangles sequentially.

Application Example: GPU Particle System

Let's introduce an application using the GPU-based Particle System: **GPUVoxelParticleSystem**.

`GPUVoxelParticleSystem` uses the `ComputeBuffer` from `GPUVoxelizer` in a Compute Shader for particle position calculations:

1. Voxelize an animated model every frame using `GPUVoxelizer`
2. Pass `GPUVoxelData`'s `ComputeBuffer` to the particle position Compute Shader
3. Render particles using GPU instancing

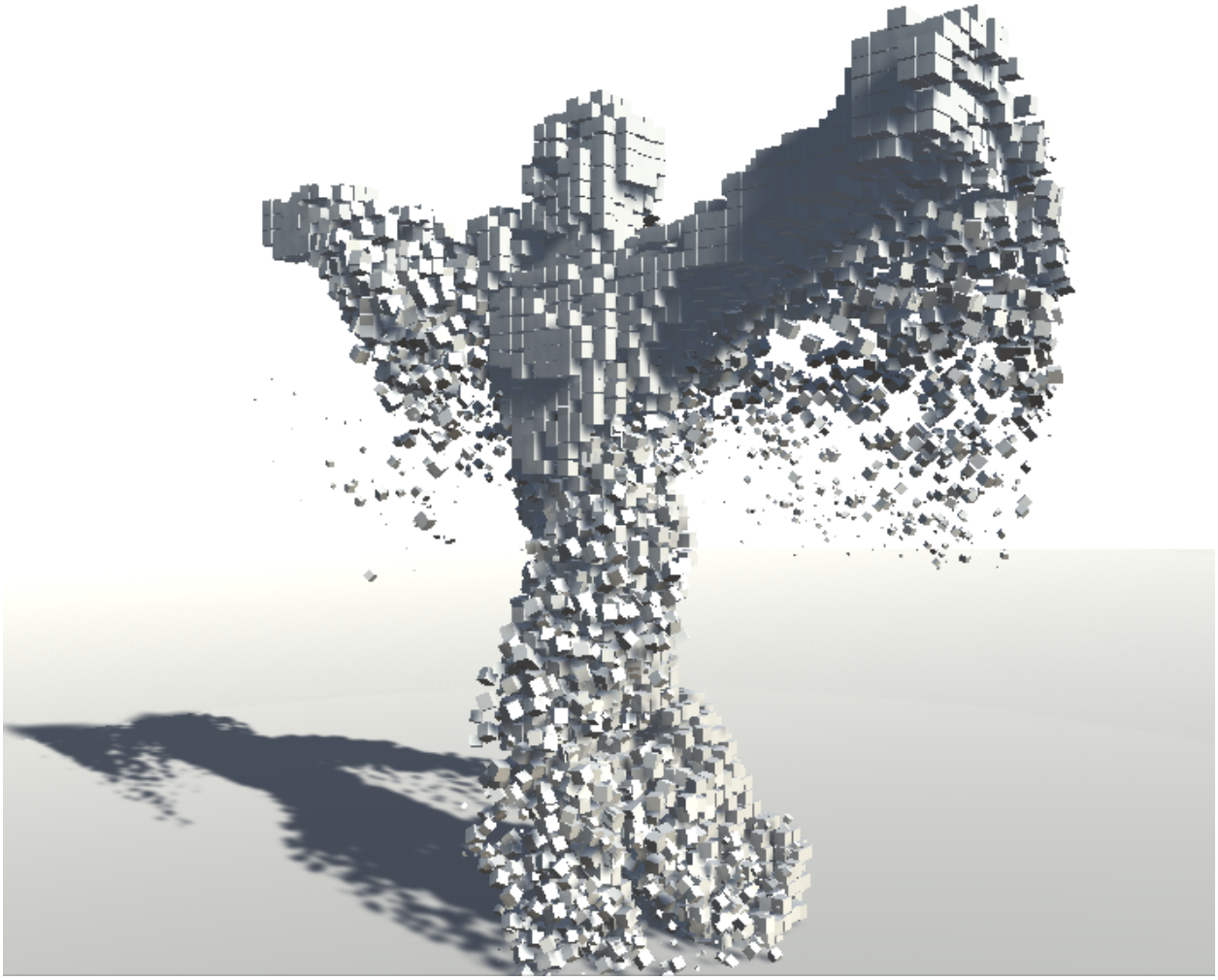


Figure: Application using GPU Particle System (GPUVoxelParticleSystem)

By spawning large numbers of particles from voxel positions, we achieve a visual of an animated model composed of particles.

Being able to voxelize animated models every frame is only possible thanks to GPU acceleration. Such high-speed GPU processing is essential for expanding the range of real-time visual expressions.

> **Real-Time Possibilities**
>
> With GPU voxelization, you can: - **Particle effects**: Characters dissolving into particles, reform from particles - **Destruction**: Convert mesh to voxels, then simulate debris - **Volume effects**: Sample voxels for volumetric fog that conforms to character shapes - **Physics**: Use voxels for simplified collision detection with thousands of objects
>
> The key is that you can update the voxel data every frame as the mesh animates!

Summary

In this chapter, we introduced the mesh voxelization algorithm using a CPU implementation, then accelerated it with a GPU implementation.

We used an approach based on triangle-voxel intersection testing. An alternative approach exists: rendering the model from XYZ directions into a 3D texture using orthographic projection.

The method presented here has challenges with texture mapping on voxelized models, but the 3D texture rendering approach might achieve coloring more easily and accurately.

Key Takeaways

What You Should Remember

1. **Voxels are 3D pixels** - basic units in a regular 3D grid, useful for volumetric data and effects
2. **SAT (Separating Axis Theorem)** is a general algorithm for convex shape intersection testing
   - For triangle-AABB: test 13 potential separating axes
   - Early exit when any axis separates the shapes
3. **Voxelization has two phases:**
   - Generate surface voxels (triangle-box intersection)
   - Fill interior voxels (ray-march along one axis)
4. **GPU parallelization strategy:**
   - Surface generation: parallelize per triangle
   - Interior fill: parallelize per XY grid cell
   - Handle race conditions by processing front/back faces separately
5. **Performance gains are dramatic** - 50x+ speedup enables real-time voxelization of animated meshes
6. **Practical applications:**
   - Particle effects driven by mesh shape
   - Dynamic destruction systems
   - Volumetric rendering
   - Collision detection optimization

References

- http://blog.wolfire.com/2009/11/Triangle-mesh-voxelization
- http://www.dyn4j.org/2010/01/sat/
- https://gdbooks.gitbooks.io/3dcollisions/content/Chapter4/aabb-triangle.html
- Game Engine Architecture, 2nd Edition, Chapter 12
- https://developer.nvidia.com/content/basics-gpu-voxelization

---

Next chapter: Explore more GPU graphics programming techniques in Unity Graphics Programming Volume 2!

Chapter 2: GPU-Based Trail

Original author: @fuqunaga Translation and annotations by Claude

---

Introduction

This chapter introduces how to create **Trails** (trajectories) using the GPU. The sample code for this chapter is “GPUBasedTrail” at: https://github.com/IndieVisualLab/UnityGraphicsProgramming2

What is a Trail?

We call the trajectory of a moving object a **Trail**. In a broad sense, this includes car tire tracks, ship wakes, and ski tracks in snow. However, in computer graphics, the most impressive trails are probably curved light effects like car taillights or homing lasers in shooting games.

What Makes Trails Special in CG

Trails create a sense of motion and history. A single particle shows where something *is*; a trail shows where it *has been*. This temporal dimension adds drama and visual interest to any moving object.

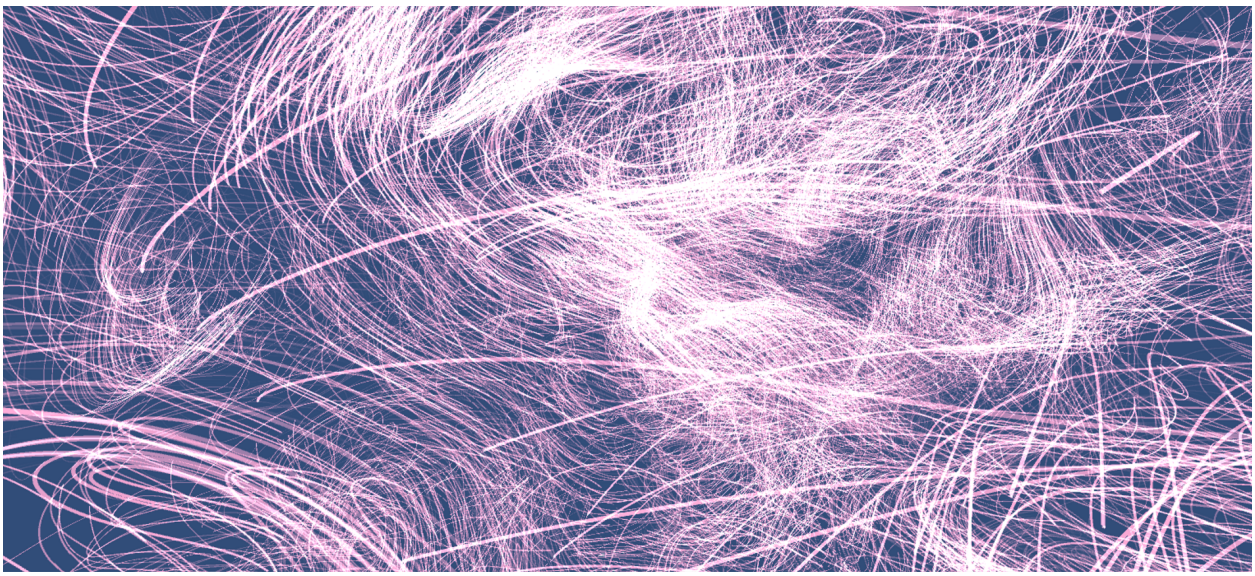
Unity’s Built-in Trail Systems

Unity provides two built-in trail types:

- **TrailRenderer** - Used to draw the trajectory of a GameObject
- **Trails module** - Used to draw particle trajectories in the Particle System

Reference links: - TrailRenderer: https://docs.unity3d.com/ja/current/Manual/class-TrailRenderer.html - Trails module: https://docs.unity3d.com/Manual/PartSysTrailsModule.html

In this chapter, we intentionally avoid using these built-in features to focus on understanding how trails work internally. By implementing on the GPU, we can achieve greater quantity than even the Trails module allows.



Sample code execution showing 10,000 trails

Why Build Our Own?

Unity’s built-in trails work great for typical use cases. But when you need: - **Massive quantity** (10,000+ trails) - **Custom behavior** (tentacles, ribbons, custom shading) - **Full GPU control** (no CPU-GPU data transfer)

…then building your own GPU-based system is the way to go.

Creating the Data

Let's start building our trail system.

Data Structure Definitions

We use three main structures:

```
// GPUTrails.cs
public struct Trail
{
    public int currentNodeIdx;
}
```

The `Trail` structure corresponds to a single trail. `currentNodeIdx` stores the index of the most recently written node in the node buffer.

```
// GPUTrails.cs
public struct Node
{
    public float time;
    public Vector3 pos;
}
```

The `Node` structure represents a control point within a trail. It stores the node's position and the time it was updated.

```
// GPUTrails.cs
public struct Input
{
    public Vector3 pos;
}
```

The `Input` structure holds one frame's worth of input from an emitter (the object leaving the trail). Here it's just position, but you could add color or other properties for interesting effects.

> **Understanding the Three Structures**
>
> | Structure | Purpose | One per… |
> |---|---|---|
> | `Trail` | Bookkeeping for one trail | Trail |
> | `Node` | One point along a trail's path | Trail segment |
> | `Input` | Current emitter position | Frame per trail |
>
> Think of it this way: an `Input` comes in each frame, gets converted to a `Node`, and the `Trail` keeps track of all its nodes.

Initialization

In `GPUTrails.Start()`, we initialize the buffers:

```
// GPUTrails.cs
trailBuffer = new ComputeBuffer(trailNum, Marshal.SizeOf(typeof(Trail)));
nodeBuffer = new ComputeBuffer(totalNodeNum, Marshal.SizeOf(typeof(Node)));
inputBuffer = new ComputeBuffer(trailNum, Marshal.SizeOf(typeof(Input)));
```

We initialize `trailBuffer` with `trailNum` elements. This means our program handles multiple trails together in batch.

The `nodeBuffer` handles all nodes for all trails in a single buffer. Indices 0 through `nodeNum−1` belong to trail 0, `nodeNum` through `2∗nodeNum−1` belong to trail 1, and so on.

The `inputBuffer` also holds `trailNum` elements, managing input for all trails.

> **Memory Layout Visualization**
>
> `nodeBuffer (if nodeNum=4, trailNum=3):`
>
> ```
> [Node0] [Node1] [Node2] [Node3] | [Node0] [Node1] [Node2] [Node3] | [Node0] [Node1] [Node2] [Node3]
> |<----- Trail 0 ----->|     |<----- Trail 1 ----->|       |<----- Trail 2 ----->|
> ```
>
> Each trail gets its own contiguous section of the node buffer.

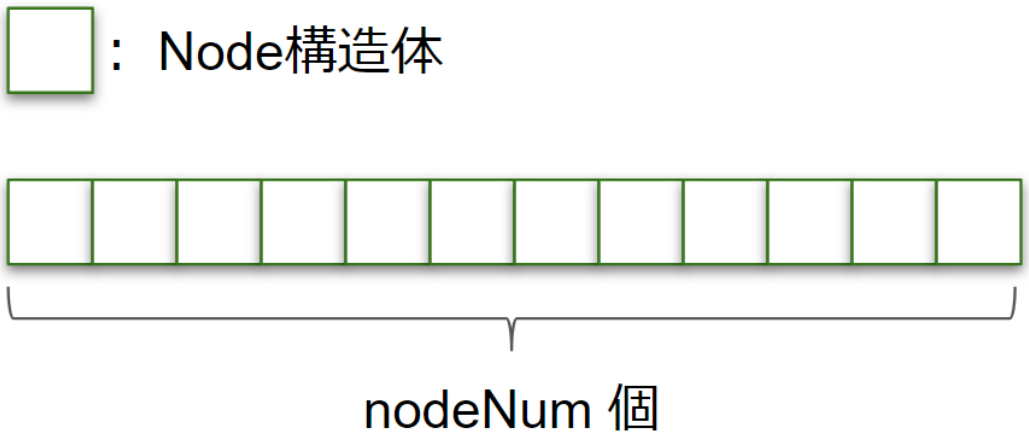
```
// GPUTrails.cs
var initTrail = new Trail() { currentNodeIdx = -1 };
var initNode = new Node() { time = -1 };

trailBuffer.SetData(Enumerable.Repeat(initTrail, trailNum).ToArray());
nodeBuffer.SetData(Enumerable.Repeat(initNode, totalNodeNum).ToArray());
```

We set initial values for each buffer. By setting `Trail.currentNodeIdx` and `Node.time` to negative values, we can later use these to determine if they're unused.

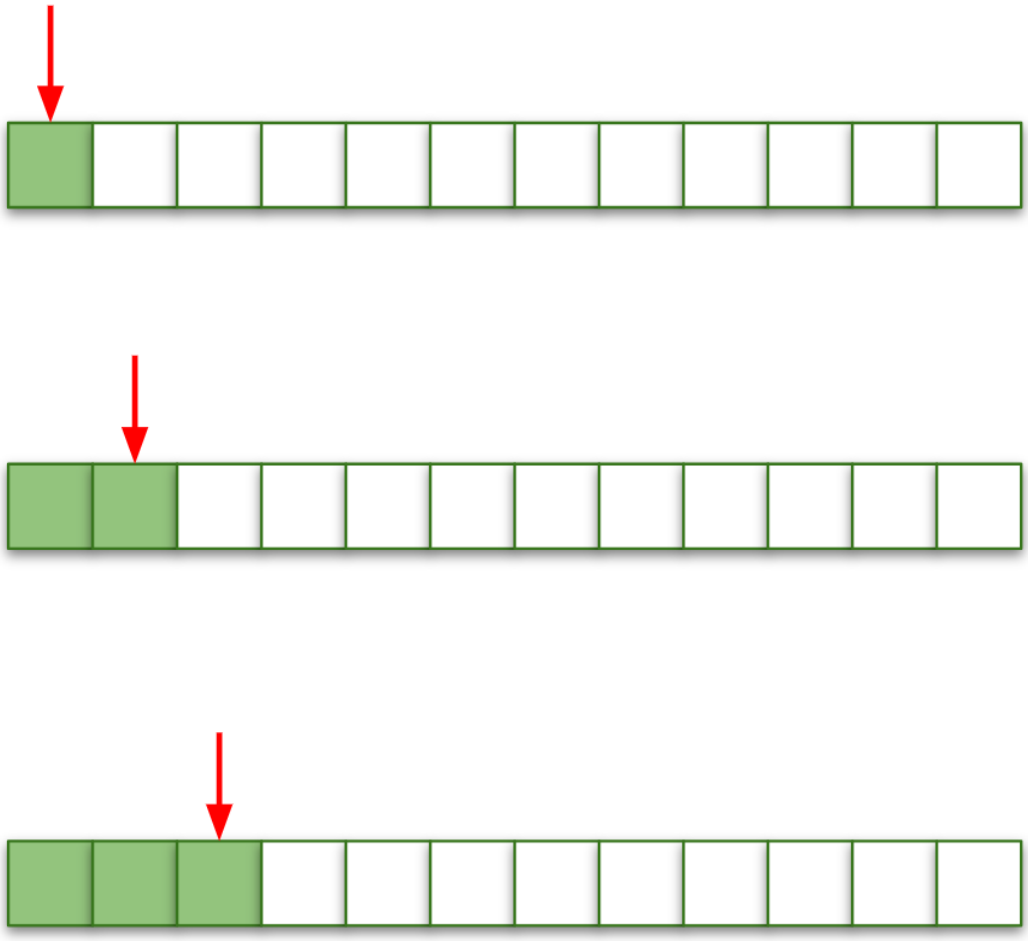
The `inputBuffer` doesn't need initialization since it will be completely written to on the first update.

Node Buffer Usage: The Ring Buffer

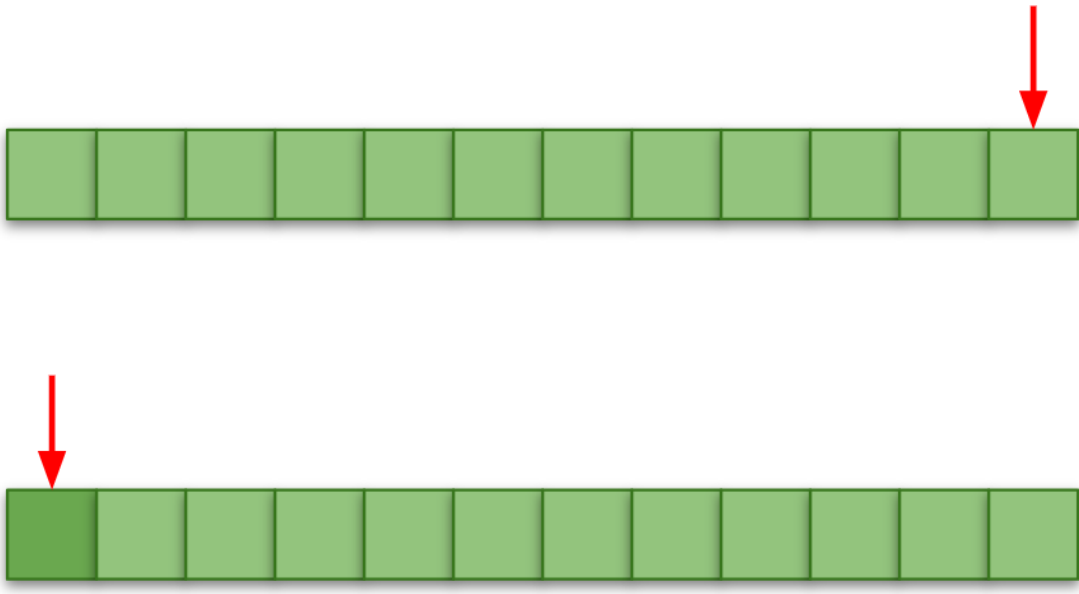
Here's how the node buffer is used:



Initial State
Nothing has been input yet



During Input
Nodes are added one by one. Some nodes are still unused.



Loop (Ring Buffer)
When all nodes are used up, we wrap around and overwrite from the beginning. This is a ring buffer.

> **Why a Ring Buffer?**
>
> A ring buffer is perfect for trails because: 1. **Fixed memory** - No allocations during runtime 2. **Automatic aging** - Old nodes naturally get overwritten 3. **Constant performance** - Same work regardless of trail length
>
> The oldest part of the trail disappears as new positions are added, creating the illusion of a trail that follows the emitter.

Input Processing

From here, we're dealing with per-frame processing. We input emitter positions and add/update nodes.

First, `inputBuffer` is updated externally. This can be done however you like. The simplest approach might be calculating on the CPU and using `ComputeBuffer.SetData()`. In the sample code, we run a simple GPU-based particle system and use those particles as emitters.

> **Column: Curl Noise**
>
> The sample code's particles move by calculating forces from Curl Noise. Curl Noise is very convenient because it can easily create pseudo-fluid-like movement. See the chapter by @sakope in this book for a detailed explanation.

Emitter Update

```
// GPUTrailParticles.cs
void Update()
{
    cs.SetInt(CSPARAM.PARTICLE_NUM, particleNum);
    cs.SetFloat(CSPARAM.TIME, Time.time);
    cs.SetFloat(CSPARAM.TIME_SCALE, _timeScale);
```

```
    cs.SetFloat(CSPARAM.POSITION_SCALE, _positionScale);
    cs.SetFloat(CSPARAM.NOISE_SCALE, _noiseScale);

    var kernelUpdate = cs.FindKernel(CSPARAM.UPDATE);
    cs.SetBuffer(kernelUpdate, CSPARAM.PARTICLE_BUFFER_WRITE, _particleBuffer);

    var updateThureadNum = new Vector3(particleNum, 1f, 1f);
    ComputeShaderUtil.Dispatch(cs, kernelUpdate, updateThureadNum);


    var kernelInput = cs.FindKernel(CSPARAM.WRITE_TO_INPUT);
    cs.SetBuffer(kernelInput, CSPARAM.PARTICLE_BUFFER_READ, _particleBuffer);
    cs.SetBuffer(kernelInput, CSPARAM.INPUT_BUFFER, trails.inputBuffer);

    var inputThreadNum = new Vector3(particleNum, 1f, 1f);
    ComputeShaderUtil.Dispatch(cs, kernelInput, inputThreadNum);
}
```

Two kernels are executed:

- **CSPARAM.UPDATE**: Updates the particles used as emitters
- **CSPARAM.WRITE_TO_INPUT**: Writes the current emitter positions to `inputBuffer`. This is used as input to the trails.

Trail Input Processing

Now let's reference `inputBuffer` and update `nodeBuffer`:

```
// GPUTrailParticles.cs
void LateUpdate()
{
    cs.SetFloat(CSPARAM.TIME, Time.time);
    cs.SetFloat(CSPARAM.UPDATE_DISTANCE_MIN, updateDistaceMin);
    cs.SetInt(CSPARAM.TRAIL_NUM, trailNum);
    cs.SetInt(CSPARAM.NODE_NUM_PER_TRAIL, nodeNum);

    var kernel = cs.FindKernel(CSPARAM.CALC_INPUT);
    cs.SetBuffer(kernel, CSPARAM.TRAIL_BUFFER, trailBuffer);
    cs.SetBuffer(kernel, CSPARAM.NODE_BUFFER, nodeBuffer);
    cs.SetBuffer(kernel, CSPARAM.INPUT_BUFFER, inputBuffer);

    ComputeShaderUtil.Dispatch(cs, kernel, new Vector3(trailNum, 1f, 1f));
}
```

On the CPU side, we just pass the necessary parameters and dispatch the compute shader. The main processing happens in the compute shader:

```
// GPUTrail.compute
[numthreads(256,1,1)]
void CalcInput (uint3 id : SV_DispatchThreadID)
{
    uint trailIdx = id.x;
    if ( trailIdx < _TrailNum)
    {
        Trail trail = _TrailBuffer[trailIdx];
        Input input = _InputBuffer[trailIdx];
```

```
        int currentNodeIdx = trail.currentNodeIdx;

        bool update = true;
        if ( trail.currentNodeIdx >= 0 )
        {
            Node node = GetNode(trailIdx, currentNodeIdx);
            float dist = distance(input.position, node.position);
            update = dist > _UpdateDistanceMin;
        }

        if ( update )
        {
            Node node;
            node.time = _Time;
            node.position = input.position;

            currentNodeIdx++;
            currentNodeIdx %= _NodeNumPerTrail;

            // write new node
            SetNode(node, trailIdx, currentNodeIdx);

            // update trail
            trail.currentNodeIdx = currentNodeIdx;
            _TrailBuffer[trailIdx] = trail;
        }
    }
}
```

Let's examine this in detail:

```
uint trailIdx = id.x;
if ( trailIdx < _TrailNum)
```

First, we use the thread ID as the trail index. Due to thread count alignment, we might get called with IDs beyond the trail count, so we filter out those cases with the `if` statement.

```
int currentNodeIdx = trail.currentNodeIdx;

bool update = true;
if ( trail.currentNodeIdx >= 0 )
{
    Node node = GetNode(trailIdx, currentNodeIdx);
    update = distance(input.position, node.position) > _UpdateDistanceMin;
}
```

Next, we check `Trail.currentNodeIdx`. A negative value indicates an unused trail.

`GetNode()` is a function that retrieves a specific node from `_NodeBuffer`. The index calculation is error-prone, so it's encapsulated in a function.

For trails already in use, we compare the distance between the latest node and the input position. If farther than `_UpdateDistanceMin`, we update; if closer, we skip.

> **Why Skip Close Inputs?**
>
> This is a crucial optimization! When an emitter is nearly stationary, it produces many inputs at almost the same position. If we recorded all of these: - We'd waste nodes on redundant data -

Adjacent nodes would have wildly varying directions (jittery) - The trail would look “noisy” and ugly

By requiring a minimum distance, we ensure nodes represent meaningful movement.

```
// GPUTrail.compute
if ( update )
{
    Node node;
    node.time = _Time;
    node.position = input.position;

    currentNodeIdx++;
    currentNodeIdx %= _NodeNumPerTrail;

    // write new node
    SetNode(node, trailIdx, currentNodeIdx);

    // update trail
    trail.currentNodeIdx = currentNodeIdx;
    _TrailBuffer[trailIdx] = trail;
}
```

Finally, we update `_NodeBuffer` and `_TrailBuffer`. The trail stores the written node’s index as `currentNodeIdx`. When the index exceeds the node count per trail, we wrap back to zero for ring buffer behavior.

The node stores the input’s time and position.

That completes the logical trail processing. Now let’s look at the rendering.

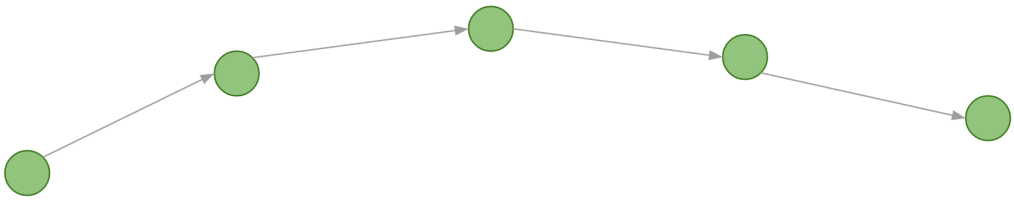
Rendering

Trail rendering is essentially connecting nodes with lines. Here we’ll keep individual trails simple and focus on quantity. To minimize polygon count, we generate billboard polygons that face the camera.

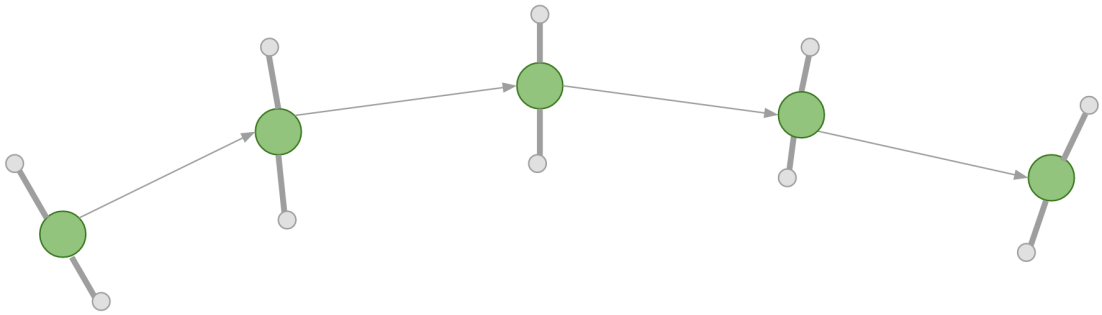
Generating Camera-Facing Billboard Polygons

Here’s how to generate camera-facing billboard polygons:

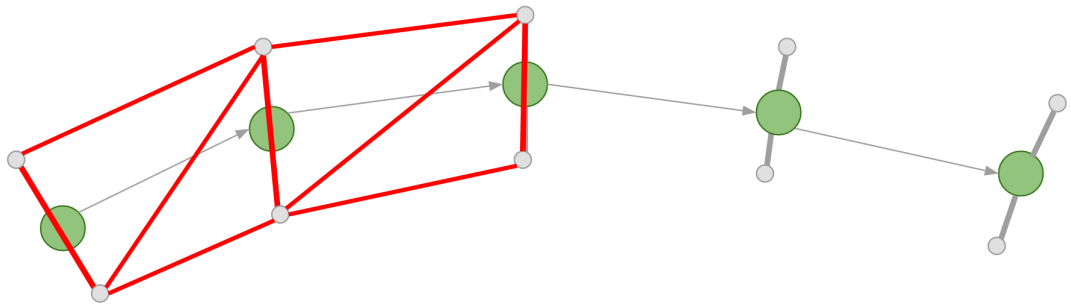
: Node



Starting with a series of nodes



From each node, calculate vertices displaced perpendicular to the view direction



Connect the generated vertices to form polygons

> **Why Billboards?**
>
> A billboard is a polygon that always faces the camera. For trails: - **Minimum geometry** - Just 2 triangles (1 quad) per segment - **Always visible** - Never edge-on to the camera - **Cheap** - No complex mesh generation
>
> The "width" of the trail is created by offsetting vertices perpendicular to both the trail direction AND the camera direction.

Let's examine the actual code.

CPU Side

The CPU side simply passes parameters to the material and calls `DrawProcedural()`:

```
// GPUTrailRenderer.cs
void OnRenderObject()
{
    _material.SetInt(GPUTrails.CSPARAM.NODE_NUM_PER_TRAIL, trails.nodeNum);
    _material.SetFloat(GPUTrails.CSPARAM.LIFE, trails._life);
    _material.SetBuffer(GPUTrails.CSPARAM.TRAIL_BUFFER, trails.trailBuffer);
    _material.SetBuffer(GPUTrails.CSPARAM.NODE_BUFFER, trails.nodeBuffer);
    _material.SetPass(0);

    var nodeNum = trails.nodeNum;
    var trailNum = trails.trailNum;
    Graphics.DrawProcedural(MeshTopology.Points, nodeNum, trailNum);
}
```

A new parameter `trails._life` appears here. This is the node lifespan - compared with each node's creation time, it's used to fade nodes to transparent as they age. This creates a smooth fading effect at the trail's tail.

Since there's no mesh or polygons to input, we use `Graphics.DrawProcedural()` to issue a draw command for `trails.nodeNum` vertices across `trails.trailNum` instances.

> **Understanding DrawProcedural**

`Graphics.DrawProcedural(MeshTopology.Points, nodeNum, trailNum)` means: - **Points topology** - Each “vertex” is processed independently - **nodeNum vertices** - One per node in a trail - **trailNum instances** - Draw this many trails

The geometry shader will expand each point into a quad. This is incredibly efficient for generating large amounts of geometry on the GPU.

GPU Side

Vertex Shader

```
// GPUTrails.shader
vs_out vert (uint id : SV_VertexID, uint instanceId : SV_InstanceID)
{
    vs_out Out;
    Trail trail = _TrailBuffer[instanceId];
    int currentNodeIdx = trail.currentNodeIdx;

    Node node0 = GetNode(instanceId, id-1);
    Node node1 = GetNode(instanceId, id); // current
    Node node2 = GetNode(instanceId, id+1);
    Node node3 = GetNode(instanceId, id+2);

    bool isLastNode = (currentNodeIdx == (int)id);

    if ( isLastNode || !IsValid(node1))
    {
        node0 = node1 = node2 = node3 = GetNode(instanceId, currentNodeIdx);
    }

    float3 pos1 = node1.position;
    float3 pos0 = IsValid(node0) ? node0.position : pos1;
    float3 pos2 = IsValid(node2) ? node2.position : pos1;
    float3 pos3 = IsValid(node3) ? node3.position : pos2;

    Out.pos = float4(pos1, 1);
    Out.posNext = float4(pos2, 1);

    Out.dir = normalize(pos2 - pos0);
    Out.dirNext = normalize(pos3 - pos1);

    float ageRate = saturate((_Time.y - node1.time) / _Life);
    float ageRateNext = saturate((_Time.y - node2.time) / _Life);
    Out.col = lerp(_StartColor, _EndColor, ageRate);
    Out.colNext = lerp(_StartColor, _EndColor, ageRateNext);

    return Out;
}
```

The vertex shader outputs information for the current node and the next node corresponding to this thread.

```
// GPUTrails.shader
Node node0 = GetNode(instanceId, id-1);
Node node1 = GetNode(instanceId, id); // current
Node node2 = GetNode(instanceId, id+1);
Node node3 = GetNode(instanceId, id+2);
```

With node1 as the current node, we reference four nodes total: one before (node0), one ahead (node2), and two ahead (node3).

Why Four Nodes?

We need four nodes to calculate smooth tangent directions: - pos0 –> pos2 gives the tangent at pos1 - pos1 –> pos3 gives the tangent at pos2

This “central difference” approach gives smoother curves than just using adjacent nodes.

```
// GPUTrails.shader
bool isLastNode = (currentNodeIdx == (int)id);

if ( isLastNode || !IsValid(node1))
{
    node0 = node1 = node2 = node3 = GetNode(instanceId, currentNodeIdx);
}
```

If the current node is at the end, or if it hasn’t been input yet, we copy all nodes from the tail node. In other words, we’re “folding” nodes that don’t have information yet onto the tail. This allows us to pass them through the polygon generation process unchanged.

```
// GPUTrails.shader
float3 pos1 = node1.position;
float3 pos0 = IsValid(node0) ? node0.position : pos1;
float3 pos2 = IsValid(node2) ? node2.position : pos1;
float3 pos3 = IsValid(node3) ? node3.position : pos2;

Out.pos = float4(pos1, 1);
Out.posNext = float4(pos2, 1);
```

We extract positions from the four nodes. Nodes other than the current one (node1) might be uninput, so we need to be careful. The case where node0 is uninput might be surprising, but when currentNodeIdx == 0, going back through the ring buffer means node0 points to the last node in the buffer, which could be unused. In this case, we also copy node1’s position to fold it to the same location. The same applies to node2 and node3.

Of these, we output pos1 and pos2 to the geometry shader.

```
// GPUTrails.shader
Out.dir = normalize(pos2 - pos0);
Out.dirNext = normalize(pos3 - pos1);
```

We also output the direction vector from pos0 to pos2 as the tangent at pos1, and the direction vector from pos1 to pos3 as the tangent at pos2.

```
// GPUTrails.shader
float ageRate = saturate((_Time.y - node1.time) / _Life);
float ageRateNext = saturate((_Time.y - node2.time) / _Life);
Out.col = lerp(_StartColor, _EndColor, ageRate);
Out.colNext = lerp(_StartColor, _EndColor, ageRateNext);
```

Finally, we calculate colors by comparing node1 and node2’s write times with the current time.

Geometry Shader

```
// GPUTrails.shader
[maxvertexcount(4)]
void geom (point vs_out input[1], inout TriangleStream<gs_out> outStream)
{
```

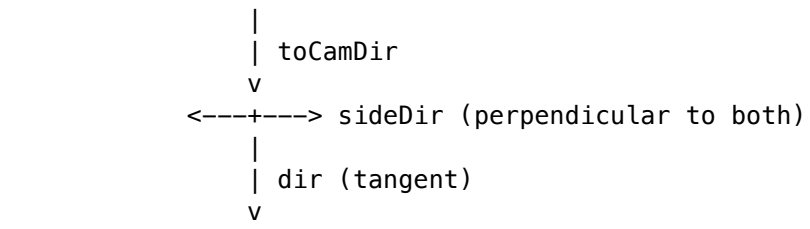
```
    gs_out output0, output1, output2, output3;
    float3 pos = input[0].pos;
    float3 dir = input[0].dir;
    float3 posNext = input[0].posNext;
    float3 dirNext = input[0].dirNext;

    float3 camPos = _WorldSpaceCameraPos;
    float3 toCamDir = normalize(camPos - pos);
    float3 sideDir = normalize(cross(toCamDir, dir));

    float3 toCamDirNext = normalize(camPos - posNext);
    float3 sideDirNext = normalize(cross(toCamDirNext, dirNext));
    float width = _Width * 0.5;

    output0.pos = UnityWorldToClipPos(pos + (sideDir * width));
    output1.pos = UnityWorldToClipPos(pos - (sideDir * width));
    output2.pos = UnityWorldToClipPos(posNext + (sideDirNext * width));
    output3.pos = UnityWorldToClipPos(posNext - (sideDirNext * width));

    output0.col =
    output1.col = input[0].col;
    output2.col =
    output3.col = input[0].colNext;

    outStream.Append (output0);
    outStream.Append (output1);
    outStream.Append (output2);
    outStream.Append (output3);

    outStream.RestartStrip();
}
```

The geometry shader is where we finally generate polygons. From the two nodes' worth of information passed from the vertex shader, we calculate four positions (a quad) and output them as a `TriangleStream`.

```
// GPUTrails.shader
float3 camPos = _WorldSpaceCameraPos;
float3 toCamDir = normalize(camPos - pos);
float3 sideDir = normalize(cross(toCamDir, dir));
```

We take the cross product of the direction vector from `pos` to camera (`toCamDir`) and the tangent vector (`dir`). This gives us the `sideDir` - the direction to expand the line width.

The Cross Product Magic



`cross(toCamDir, dir)` gives a vector perpendicular to both - exactly what we need to create width while facing the camera!

```
// GPUTrails.shader
output0.pos = UnityWorldToClipPos(pos + (sideDir * width));
output1.pos = UnityWorldToClipPos(pos - (sideDir * width));
```

We calculate vertices displaced in positive and negative `sideDir` directions. Here we also transform to clip space coordinates for passing to the fragment shader. Applying the same process to `posNext` gives us all four vertices.

```
// GPUTrails.shader
output0.col =
output1.col = input[0].col;
output2.col =
output3.col = input[0].colNext;
```

We assign colors to each vertex and we're done.

Fragment Shader

```
// GPUTrails.shader
fixed4 frag (gs_out In) : COLOR
{
    return In.col;
}
```

Finally, the fragment shader. It couldn't be simpler - we just output the color!

Applications

With this, you should be able to generate trails. While we only used color in this example, you could apply textures, vary the width, and explore many other variations.

Also, as the source code is separated into `GPUTrails.cs` and `GPUTrailRenderer.cs`, the shader side (`GPUTrails.shader`) simply reads buffers and draws. As long as you prepare `_TrailBuffer` and `_NodeBuffer`, this can actually be repurposed for any line-based rendering, not just trails.

While this example only adds to `_NodeBuffer`, by updating all nodes every frame, you could express things like wriggling tentacles.

Ideas for Extensions

- **Textures**: Add UV coordinates and sample a texture for glow effects
- **Variable width**: Make trails wider at the head, thinner at the tail
- **Per-node color**: Store color in each node for rainbow trails
- **Tentacles**: Update all nodes with sine waves for organic movement
- **Ribbons**: Use the cross product of adjacent tangents for ribbon orientation
- **Lightning**: Add random offsets to nodes for electrical effects

---

Summary

This chapter introduced a minimal GPU implementation of trails. Using the GPU makes debugging harder, but enables overwhelming quantity that would be impossible on the CPU. I hope this book helps even one more person experience that "Wow!" feeling.

Trails occupy an interesting space between “displaying models” and “screen-space algorithmic rendering,” offering wide application possibilities. The understanding gained through this process should be useful for programming various visual expressions beyond just trails.

Key Takeaways

What You Should Remember

1. **Trail Data Architecture**
   - `Trail`: Tracks which node is current (bookkeeping)
   - `Node`: Stores position + timestamp (the actual trail points)
   - `Input`: Per-frame emitter position (what feeds the trail)
2. **Ring Buffer for Nodes**
   - Fixed memory allocation, no runtime allocations
   - Old nodes automatically overwritten
   - `currentNodeIdx % nodeNum` handles wraparound
3. **Minimum Distance Threshold**
   - Skip nodes when emitter barely moves
   - Prevents jittery trails from near-stationary objects
   - Critical for visual quality
4. **Billboard Rendering Pipeline**
   - Vertex shader: Prepare 4 nodes for tangent calculation
   - Geometry shader: Generate camera-facing quads
   - Fragment shader: Just output interpolated color
5. **The Cross Product for Billboards**
   - `cross(toCamera, tangent)` = perpendicular side direction
   - Offset vertices by this to create trail width
   - Always faces camera regardless of viewing angle
6. **GPU Advantages**
   - 10,000+ trails at interactive framerates
   - All computation stays on GPU (no CPU-GPU transfers for rendering)
   - Easily extensible for custom effects

Performance Considerations

- **Node count per trail**: More nodes = longer trails but more memory/computation
- **Update distance minimum**: Higher = fewer nodes, better performance, but choppier trails
- **Geometry shader**: Can be a bottleneck; consider compute shader + indirect draw for extreme cases

---

Next chapter: Explore more advanced GPU techniques for particle systems and effects!

Chapter 3: Geometry Shader Applications for Line Expressions

Author: kaiware007

Sample Project: “GeometryWireframe” in the Unity Graphics Programming 2 Repository

---

Introduction

This chapter explores using Geometry Shaders to render wireframe polygons - a technique used in the author's visual artwork for Art Hack Day 2018. The approach demonstrates how to dynamically generate vertices in the GPU pipeline to create line-based visual effects.

Part 1: Basic Line Drawing with Graphics.DrawProcedural

Understanding the Foundation

In Unity, lines are typically drawn using `LineRenderer` or the `GL` class. However, when expecting high draw volumes, `Graphics.DrawProcedural` offers better performance by leveraging GPU-based vertex generation.

Simple Sine Wave Example

The sample scene "SampleWaveLine" demonstrates basic line drawing:

```
[ExecuteInEditMode]
public class RenderWaveLine : MonoBehaviour {
    [Range(2,50)]
    public int vertexNum = 4;
    public Material material;

    private void OnRenderObject()
    {
        material.SetInt("_VertexNum", vertexNum - 1);
        material.SetPass(0);
        Graphics.DrawProcedural(MeshTopology.LineStrip, vertexNum);
    }
}
```

Key Points: - `Graphics.DrawProcedural` executes immediately, so it must be called within `OnRender-Object` - `OnRenderObject` is called after all cameras finish rendering the scene - The first argument is `MeshTopology` - specifying how mesh vertices are connected - Available topologies: `Triangles`, `Quads`, `Lines`, `LineStrip`, `Points`

Shader-Based Vertex Generation

The clever aspect of this approach is that vertex positions are calculated entirely in the shader:

```
v2f vert (uint id : SV_VertexID)
{
    float div = (float)id / _VertexNum;
    float4 pos = float4(
        (div - 0.5) * _ScaleX,
        sin(div * 2 * PI + _Time.y * _Speed) * _ScaleY,
        0, 1);

    v2f o;
    o.vertex = UnityObjectToClipPos(pos);
    return o;
}
```

How It Works: - `SV_VertexID` provides a unique vertex index (0 to vertexCount-1) - Dividing by vertex count gives a 0-1 ratio - This ratio drives the sine wave calculation - No vertex data needs to be passed from C# - it's all computed on the GPU

Part 2: Drawing 2D Polygons with Geometry Shaders

The Geometry Shader Concept

Geometry Shaders sit between Vertex and Fragment shaders, with the unique ability to **generate new vertices**. From a single input point, we can create an entire polygon.

Implementation

The C# side is minimal:

```
[ExecuteInEditMode]
public class SinglePolygon2D : MonoBehaviour {
    [Range(2, 64)]
    public int vertexNum = 3;
    public Material material;

    private void OnRenderObject()
    {
        material.SetInt("_VertexNum", vertexNum);
        material.SetMatrix("_TRS", transform.localToWorldMatrix);
        material.SetPass(0);
        Graphics.DrawProcedural(MeshTopology.Points, 1);
    }
}
```

Notable Changes: - `MeshTopology.Points` is used (only 1 vertex required) - _TRS matrix enables transform control via the GameObject

The Geometry Shader

```
#pragma geometry geom

[maxvertexcount(65)]
void geom(point Output input[1], inout LineStream<Output> outStream)
{
    Output o;
    float rad = 2.0 * PI / (float)_VertexNum;
    float time = _Time.y * _Speed;
    float4 pos;

    for (int i = 0; i <= _VertexNum; i++) {
        pos.x = cos(i * rad + time) * _Scale;
        pos.y = sin(i * rad + time) * _Scale;
        pos.z = 0;
        pos.w = 1;
        o.pos = UnityObjectToClipPos(pos);
        outStream.Append(o);
    }
```

```
    outStream.RestartStrip();
}
```

Key Attributes:

Attribute	Description
`[maxvertexcount(65)]`	Maximum vertices the shader can output (64 polygon vertices + 1 to close)
`point Output input[1]`	Input from vertex shader - `point` means 1 vertex, use `triangle` for mesh work
`inout LineStream<Output>`	Output type - also available: `PointStream`, `TriangleStream`

Stream Operations: - `Append()` - adds a vertex to the current line strip - `RestartStrip()` - ends current strip, next `Append()` starts a new disconnected line

---

Part 3: Creating an Octahedron Sphere

What is an Octahedron Sphere?

A regular octahedron consists of 8 equilateral triangles. An Octahedron Sphere is created by subdividing these triangles using **spherical linear interpolation (slerp)**, causing vertices to curve outward into a sphere shape.

Unlike linear interpolation (straight line between points), spherical interpolation follows an arc along a sphere's surface.

The Subdivision Algorithm

The algorithm works on each of the 8 triangular faces:

1. **Define Initial Vertices**: Store the 24 vertices of the normalized octahedron (8 triangles x 3 vertices)
2. **Iterate Subdivision Levels**: For each level n:
    - Divide each edge into n segments using slerp
    - Create new triangles from the subdivided points
3. **Spherical Linear Interpolation (Slerp)**:

```
float4 qslerp(float4 a, float4 b, float t)
{
    float4 r;
    float t_ = 1 - t;
    float wa, wb;
    float theta = acos(a.x * b.x + a.y * b.y + a.z * b.z + a.w * b.w);
    float sn = sin(theta);
    wa = sin(t_ * theta) / sn;
    wb = sin(t * theta) / sn;
    r.x = wa * a.x + wb * b.x;
    r.y = wa * a.y + wb * b.y;
    r.z = wa * a.z + wb * b.z;
    r.w = wa * a.w + wb * b.w;
    normalize(r);
```

```
    return r;
}
```

Triangle Subdivision Flow (n=2 Example)

Step 1: Calculate edge points using slerp ratios

```
float4 edge_p1 = qslerp(init_vectors[i], init_vectors[i + 2], (float)p / n);
float4 edge_p2 = qslerp(init_vectors[i + 1], init_vectors[i + 2], (float)p / n);
float4 edge_p3 = qslerp(init_vectors[i], init_vectors[i + 2], (float)(p + 1) / n);
float4 edge_p4 = qslerp(init_vectors[i + 1], init_vectors[i + 2], (float)(p + 1) / n);
```

Step 2: Calculate intermediate vertices (a, b, c, d) by interpolating between edge points

Step 3: Output triangles (a,b,c) and (c,b,d) using the stream

---

Bonus Samples

The repository includes additional advanced examples:

1. **GPU Instancing Version** - SampleOctahedronSphereInstancing scene
2. **High Subdivision (9 levels)** - SampleOctahedronSphereMultiVertex scene
3. **High Subdivision + Instancing** - SampleOctahedronSphereMultiVertexInstancing scene

---

Key Takeaways

1. **Geometry Shaders enable dynamic vertex generation** - Create complex geometry from minimal input data
2. **`Graphics.DrawProcedural` bypasses mesh overhead** - Ideal for procedural line/point rendering
3. **`SV_VertexID` enables shader-only vertex calculation** - No CPU-GPU data transfer needed
4. **Stream primitives (`LineStream`, `TriangleStream`)** control output geometry type
5. **Spherical interpolation creates smooth spherical surfaces** - Essential for geodesic sphere generation
6. **Geometry Shaders have vertex limits** - Plan `maxvertexcount` carefully for complex geometry

---

Applications

While Geometry Shaders are commonly used for: - Polygon subdivision - Particle billboard generation

The techniques in this chapter open possibilities for: - Dynamic wireframe visualizations - Procedural geometric patterns - Real-time generative art - Efficient line-based visual effects

Experiment with these foundations to discover new visual expressions!

Chapter 4: Projection Spray - Custom Lighting and Texture Painting

Author: Sugino Hironori

Sample Project: “ProjectionSpray” folder in the Unity Graphics Programming 2 Repository

Introduction

This chapter covers implementing custom lighting without Unity's built-in lights, then applies those techniques to create a real-time spray painting system for 3D objects. The concept is to learn from Unity's built-in shaders (UnityCG.cginc) and apply that knowledge to create new functionality.

Part 1: Implementing Custom Lights

Understanding Unity's Built-in Lighting

Unity's built-in shaders can be downloaded from the Unity Download Archive. Key lighting files include:

- `CGIncludes/UnityCG.cginc`
- `CGIncludes/AutoLight.cginc`
- `CGIncludes/Lighting.cginc`
- `CGIncludes/UnityLightingCommon.cginc`

Lambert Lighting Basics

The fundamental diffuse calculation in `Lighting.cginc`:

```
struct SurfaceOutput {
    fixed3 Albedo;
    fixed3 Normal;
    fixed3 Emission;
    half Specular;
    fixed Gloss;
    fixed Alpha;
};

inline fixed4 UnityLambertLight (SurfaceOutput s, UnityLight light)
{
    fixed diff = max(0, dot(s.Normal, light.dir));
    fixed4 c;
    c.rgb = s.Albedo * light.color * diff;
    c.a = s.Alpha;
    return c;
}
```

The key calculation: `dot(s.Normal, light.dir)` - the dot product between surface normal and light direction determines brightness.

Visualizing Mesh Normals

Before implementing lighting, understanding normal visualization helps (Scene: `00_viewNormal.unity`):

```
v2f vert (appdata v)
{
    v2f o;
    o.vertex = UnityObjectToClipPos(v.vertex);
    o.normal = UnityObjectToWorldNormal(v.normal);
    return o;
```

```
}

half4 frag (v2f i) : SV_Target
{
    fixed4 col = half4(i.normal, 1);
    return col;
}
```

Built-in Helper Functions: - `UnityObjectToClipPos` - transforms vertex from object to clip space - `UnityObjectToWorldNormal` - transforms normal from object to world space

---

Part 2: Point Light Implementation

Scene: `01_pointLight.unity`

C# Component

```
[ExecuteInEditMode]
public class PointLightComponent : MonoBehaviour
{
    static MaterialPropertyBlock mpb;
    public Renderer targetRenderer;
    public float intensity = 1f;
    public Color color = Color.white;

    void Update()
    {
        if (targetRenderer == null) return;
        if (mpb == null) mpb = new MaterialPropertyBlock();

        targetRenderer.GetPropertyBlock(mpb);
        mpb.SetVector("_LitPos", transform.position);
        mpb.SetFloat("_Intensity", intensity);
        mpb.SetColor("_LitCol", color);
        targetRenderer.SetPropertyBlock(mpb);
    }
}
```

Shader Implementation

```
fixed4 frag (v2f i) : SV_Target
{
    half3 to = i.worldPos - _LitPos;
    half3 lightDir = normalize(to);
    half dist = length(to);
    half atten = _Intensity * dot(-lightDir, i.normal) / (dist * dist);
    half4 col = max(0.0, atten) * _LitCol;
    return col;
}
```

Key Formula: Intensity follows inverse square law: `intensity / (distance * distance)`

Part 3: Spotlight Implementation

Scene: `02_spotLight.unity`

Added Parameters

Spotlights require additional information: - Light direction (via `worldToLocalMatrix`) - Projection matrix (field of view, range) - Light cookie texture

```
var projMatrix = Matrix4x4.Perspective(angle, 1f, 0f, range);
var worldToLightMatrix = transform.worldToLocalMatrix;

mpb.SetMatrix("_WorldToLitMatrix", worldToLightMatrix);
mpb.SetMatrix("_ProjMatrix", projMatrix);
mpb.SetTexture("_Cookie", cookie);
```

Spotlight Shader

```
fixed4 frag (v2f i) : SV_Target
{
    // Diffuse calculation
    half3 to = i.worldPos - _LitPos.xyz;
    half3 lightDir = normalize(to);
    half dist = length(to);
    half atten = _Intensity * dot(-lightDir, i.normal) / (dist * dist);

    // Spotlight cone calculation
    half4 lightSpacePos = mul(_WorldToLitMatrix, half4(i.worldPos, 1.0));
    half4 projPos = mul(_ProjMatrix, lightSpacePos);
    projPos.z *= -1;
    half2 litUv = projPos.xy / projPos.z;
    litUv = litUv * 0.5 + 0.5;

    // Cookie sampling and bounds check
    half lightCookie = tex2D(_Cookie, litUv);
    lightCookie *= 0 < litUv.x && litUv.x < 1 &&
                   0 < litUv.y && litUv.y < 1 && 0 < projPos.z;

    half4 col = max(0.0, atten) * _LitCol * lightCookie;
    return col;
}
```

Part 4: Shadow Implementation

Scene: `03_spotLight-withShadow.unity`

Creating a Depth Texture

Shadows require comparing depths from the light's perspective:

```
depthOutput = new RenderTexture(
    shadowMapResolution, shadowMapResolution, 16, RenderTextureFormat.RFloat);
depthOutput.wrapMode = TextureWrapMode.Clamp;
```

```
_c.targetTexture = depthOutput;
_c.SetReplacementShader(depthRenderShader, "RenderType");
```

Depth Render Shader

```
v2f vert (float4 pos : POSITION)
{
    v2f o;
    o.vertex = UnityObjectToClipPos(pos);
    o.depth = abs(UnityObjectToViewPos(pos).z);
    return o;
}

float frag (v2f i) : SV_Target
{
    return i.depth;
}
```

Shadow Comparison

```
// Shadow calculation
half lightDepth = tex2D(_LitDepth, litUv).r;
atten *= 1.0 - saturate(10 * abs(lightSpacePos.z) - 10 * lightDepth);
```

If `lightSpacePos.z > lightDepth`, the surface is in shadow.

Part 5: Projection Spray System

Now we apply the spotlight technique to paint on 3D objects in real-time.

UV2 for Lightmap Painting

Objects without UV data can use Unity's "Generate Lightmap UVs" option to create a second UV channel (`TEXCOORD1`) suitable for painting.

UV2 Expansion Shader

The key insight: we can render the mesh "unfolded" into UV2 space:

```
v2f vert(appdata v)
{
#if UNITY_UV_STARTS_AT_TOP
    v.uv2.y = 1.0 - v.uv2.y;
#endif
    float4 pos0 = UnityObjectToClipPos(v.vertex);
    float4 pos1 = float4(v.uv2 * 2.0 - 1.0, 0.0, 1.0);

    v2f o;
    o.vertex = lerp(pos0, pos1, _T);  // Animate between 3D and UV2
    o.uv2 = v.uv2;
    o.worldPos = mul(unity_ObjectToWorld, v.vertex).xyz;
    o.normal = UnityObjectToWorldNormal(v.normal);
```

```
    return o;
}
```

The Spray System Architecture

Components:

1. **ProjectionSpray.cs** - Spray position/settings
2. **Drawable.cs** - Object being painted (holds RenderTexture)
3. **DrawableController.cs** - Orchestrates the painting

Drawable Component (Ping-Pong Buffer)

```
public void Draw(Material drawingMat)
{
    drawingMat.SetTexture("_MainTex", pingPongRts[0]);

    var currentActive = RenderTexture.active;
    RenderTexture.active = pingPongRts[1];
    GL.Clear(true, true, Color.clear);
    drawingMat.SetPass(0);
    Graphics.DrawMeshNow(mesh, transform.localToWorldMatrix);
    RenderTexture.active = currentActive;

    Swap(pingPongRts);

    // Optional crack filling
    if(fillCrack != null)
    {
        Graphics.Blit(pingPongRts[0], pingPongRts[1], fillCrack);
        Swap(pingPongRts);
    }

    Graphics.CopyTexture(pingPongRts[0], output);
}
```

Spray Painting Shader

```
half4 frag (v2f i) : SV_Target
{
    // Spotlight-style calculations for spray intensity
    half3 to = i.worldPos - _DrawerPos.xyz;
    half3 dir = normalize(to);
    half dist = length(to);
    half atten = _Emission * dot(-dir, i.normal) / (dist * dist);

    // Cookie and shadow sampling (same as spotlight)
    // ...

    // Blend existing color with spray color
    i.uv.y = 1 - i.uv.y;
    half4 col = tex2D(_MainTex, i.uv);
    col.rgb = lerp(col.rgb, _Color.rgb,
                   saturate(col.a * _Emission * atten * cookie));
```

```
    col.a = 1;
    return col;
}
```

Key Takeaways

Concept	Application
Lambert Lighting	`dot(normal, lightDir)` for basic diffuse
Inverse Square Law	`intensity / (distance * distance)` for realistic falloff
Projection Matrix	Converts world space to light/projector space
Depth Comparison	Enables shadow mapping
UV2 Space Rendering	Allows texture painting via mesh rendering
Ping-Pong Buffers	Enables iterative texture updates

Summary

By studying Unity's built-in shader includes (`UnityCG.cginc`, `Lighting.cginc`), we can:

1. Understand how Unity implements standard lighting
2. Create custom light systems independent of Unity's lighting
3. Repurpose lighting math for creative applications like texture painting

The projection spray technique demonstrates how understanding fundamentals opens doors to novel applications - the same math that creates spotlight shadows can paint colors onto 3D surfaces in real-time.

References

- Unity Built-in Shader Helper Functions: Unity Manual
- Coordinate transformation details: Unity Graphics Programming vol.1, Chapter 9 "Multi Plane Perspective Projection"

Chapter 5: Introduction to Procedural Noise

Author: Oishi

Sample Project: "TheStudyOfProceduralNoise" in the Unity Graphics Programming 2 Repository

---

Introduction

This chapter provides a comprehensive explanation of noise algorithms used in computer graphics. Noise was developed in the 1980s as a new technique for procedural texture generation. Early computers had very limited memory, making stored texture images impractical. Procedural noise offered an elegant solution.

Natural phenomena like terrain, clouds, water, fire, marble, wood grain, and crystals exhibit both visual complexity and regular patterns. Noise can generate texture patterns optimal for representing these natural elements, making it an indispensable technique for procedural graphics.

The most notable noise algorithms are **Perlin Noise** and **Simplex Noise**, both achievements of **Ken Perlin**.

What is Noise?

In computer graphics, noise refers to a function that takes an N-dimensional vector as input and returns a scalar value with these characteristics:

- **Continuous** - Changes smoothly between adjacent regions
- **Isotropic** - Statistically invariant under rotation (rotating a region doesn't change its characteristics)
- **Translation invariant** - Statistically invariant under translation
- **Band-limited** - Energy is concentrated in a specific frequency spectrum

Applications by Dimension

Dimensions	Input	Use Case
1D	Time	Animation
2D	UV coordinates	Textures
3D	UV + Time	Animated textures
3D	Local coordinates	Solid/volumetric textures
4D	Local coords + Time	Animated volumetric textures

Value Noise

The simplest noise algorithm, ideal for understanding the fundamentals.

Algorithm

1. Define a grid with evenly spaced lattice points
2. Calculate pseudo-random values at each lattice point
3. Interpolate values between lattice points

Implementation Details

Grid Calculation:

```
// Integer part (lattice coordinates)
float2 i = floor(v);
// Fractional part (position within cell)
float2 f = frac(v);

// Four corner coordinates
float2 i00 = i;
float2 i10 = i + float2(1.0, 0.0);
float2 i01 = i + float2(0.0, 1.0);
float2 i11 = i + float2(1.0, 1.0);
```

Pseudo-Random Function:

```
float rand(float2 co)
{
```

```
    return frac(sin(dot(co.xy, float2(12.9898, 78.233))) * 43758.5453);
}
```

Hermite Interpolation (smoother than linear):

```
// 3rd-degree Hermite curve: 3t^2 - 2t^3
float2 interpolate(float2 t)
{
    return t * t * (3.0 - 2.0 * t);
}
```

Final Interpolation:

```
float2 u = interpolate(f);
return lerp(lerp(n00, n10, u.x), lerp(n01, n11, u.x), u.y);
```

Limitation: Value Noise shows visible grid artifacts because it lacks true isotropy.

Perlin Noise (Gradient Noise)

Developed by Ken Perlin for the 1982 film “Tron” and published at SIGGRAPH 1985 in “An Image Synthesizer.”

Key Difference from Value Noise

Instead of random scalar values at lattice points, Perlin Noise uses **gradient vectors**.

Algorithm

1. Calculate lattice coordinates
2. Generate gradient vectors at each lattice point
3. Calculate vectors from lattice points to target point P
4. Compute dot products between gradients and offset vectors
5. Interpolate results using Hermite curves

The Dot Product Insight

The dot product measures how aligned two vectors are: - Same direction: +1 - Perpendicular: 0 - Opposite: -1

When the gradient aligns with the offset vector, noise values are high; when opposed, values are low.

Implementation

```
float originalPerlinNoise(float2 v)
{
    float2 i = floor(v);
    float2 f = frac(v);

    // Lattice corner coordinates
    float2 i00 = i;
    float2 i10 = i + float2(1.0, 0.0);
    float2 i01 = i + float2(0.0, 1.0);
    float2 i11 = i + float2(1.0, 1.0);
```

```
    // Offset vectors from corners to point
    float2 p00 = f;
    float2 p10 = f - float2(1.0, 0.0);
    float2 p01 = f - float2(0.0, 1.0);
    float2 p11 = f - float2(1.0, 1.0);

    // Gradient vectors (normalized random)
    float2 g00 = normalize(pseudoRandom(i00));
    float2 g10 = normalize(pseudoRandom(i10));
    float2 g01 = normalize(pseudoRandom(i01));
    float2 g11 = normalize(pseudoRandom(i11));

    // Dot products
    float n00 = dot(g00, p00);
    float n10 = dot(g10, p10);
    float n01 = dot(g01, p01);
    float n11 = dot(g11, p11);

    // Interpolation
    float2 u_xy = interpolate(f.xy);
    float2 n_x = lerp(float2(n00, n01), float2(n10, n11), u_xy.x);
    return lerp(n_x.x, n_x.y, u_xy.y);
}
```

Improved Perlin Noise

Published by Ken Perlin in 2001 to address two issues in the original algorithm.

Improvement 1: 5th-Degree Hermite Curve

The original 3rd-degree curve has discontinuous second derivatives at cell boundaries, causing visual artifacts in bump mapping.

5th-Degree Hermite Curve:

```
f(t) = 6t^5 - 15t^4 + 10t^3
```

This ensures both first and second derivatives are zero at t=0 and t=1, creating smooth continuity.

Improvement 2: Constrained Gradient Vectors

In 3D, random gradients can cluster along axes, causing spots. The improved version uses exactly 12 gradient directions:

```
(1,1,0), (-1,1,0), (1,-1,0), (-1,-1,0),
(1,0,1), (-1,0,1), (1,0,-1), (-1,0,-1),
(0,1,1), (0,-1,1), (0,1,-1), (0,-1,-1)
```

These point to the edges of a cube, providing good distribution while simplifying dot product calculations.

Simplex Noise

Also by Ken Perlin (2001), designed as a superior alternative to classic Perlin Noise.

Advantages Over Perlin Noise

Aspect	Perlin Noise	Simplex Noise
Complexity	O(2^N)	O(N^2)
Higher dimensions	Exponential cost increase	Polynomial cost increase
Visual artifacts	Some directional bias	Minimal artifacts
Hardware efficiency	Moderate	Excellent

The Simplex Grid

Instead of hypercubes (squares, cubes), Simplex Noise uses **simplices** - the simplest shapes that tile N-dimensional space: - 1D: Line segments - 2D: Equilateral triangles - 3D: Tetrahedra - 4D: Pentatopes (5-cell)

An N-dimensional simplex has only N+1 vertices, compared to 2^N for a hypercube.

Determining Which Simplex Contains Point P

1. **Skew** the input space to align simplices with a regular grid
2. **Floor** to find which simplex unit cell
3. **Compare** coordinate magnitudes to determine which simplex within the unit

Summation Instead of Interpolation

Rather than interpolating between corners, Simplex Noise sums contributions from each corner, using a radially-decaying influence function:

```
Each corner's contribution = gradient_extrapolation * radial_falloff
Total = sum of all corner contributions
```

Implementation Highlights

Permutation Polynomial (replaces lookup tables):

```
// Generates pseudo-random indices without tables
float3 permute(float3 x)
{
    return fmod(((x * 34.0) + 1.0) * x, 289.0);
}
```

Cross-Polytope Gradients:

Gradients are distributed on the surface of: - 2D: Square (diamond orientation) - 3D: Octahedron - 4D: 16-cell (truncated)

Taylor Series Normalization:

```
// Fast inverse square root approximation
float3 taylorInvSqrt(float3 r)
{
    return 1.79284291400159 - 0.85373472095314 * r;
}
```

Sample Code Locations

```
TheStudyOfProceduralNoise/
├── Scenes/
│   ├── ShaderExampleList    # View all noise types
│   └── CompareBumpmap       # Compare interpolation curves
└── Shaders/ProceduralNoise/
    ├── ValueNoise2D.cginc
    ├── ValueNoise3D.cginc
    ├── OriginalPerlinNoise2D.cginc
    ├── ClassicPerlinNoise2D.cginc  # Improved Perlin
    ├── SimplexNoise2D.cginc
    └── [3D and 4D variants...]
```

Key Takeaways

1. **Value Noise** is simple but shows grid artifacts
2. **Perlin Noise** uses gradients for isotropy
3. **Improved Perlin** fixes bump mapping artifacts with 5th-degree curves
4. **Simplex Noise** scales better to higher dimensions
5. **All noise types** share the pattern: lattice + randomness + interpolation
6. **Performance matters** - noise runs per-pixel, so efficiency is crucial

---

References

1. “An Image Synthesizer” - Ken Perlin, SIGGRAPH 1985
2. “Improving Noise” - Ken Perlin, 2002
3. “Simplex noise demystified” - Stefan Gustavson, 2005
4. “Efficient computational noise in GLSL” - McEwan et al., 2012
5. Reference implementation: http://mrl.nyu.edu/~perlin/noise/

The next chapter demonstrates practical noise application with Curl Noise for pseudo-fluid effects.

Chapter 6: Curl Noise - Pseudo-Fluid Noise Algorithm

Author: Sakota

Sample Project: “CurlNoise” in the Unity Graphics Programming 2 Repository

---

Introduction

This chapter explains the GPU implementation of **Curl Noise**, a pseudo-fluid algorithm. Curl Noise provides fluid-like motion at significantly lower computational cost than full fluid simulations like Navier-Stokes equations.

With the increasing need for real-time rendering at 4K and 8K resolutions, lightweight algorithms like Curl Noise become valuable choices for achieving fluid-like effects on lower-spec machines or at high resolutions.

What is Curl Noise?

Curl Noise was published in 2007 by Professor Robert Bridson of the University of British Columbia (also known for the FLIP fluid simulation method). While previous volumes covered Navier-Stokes fluid simulation, Curl Noise offers a pseudo-fluid alternative with much lower computational requirements.

Understanding Velocity Fields

Fluid simulation fundamentally requires a **velocity field** - a vector field where each point in space has an associated velocity vector.

2D Velocity Field Visualization

Imagine a 2D plane where each small region has its own direction and magnitude arrow. In 3D, picture a cube subdivided into tiny blocks, each with its own velocity vector.

The Curl Noise Algorithm

The Core Insight

Curl Noise cleverly uses **gradient noise** (like Perlin or Simplex Noise) as a **potential field**, then derives a velocity field from it. This chapter uses 3D Simplex Noise as the potential field.

The Mathematical Formula

The Curl Noise algorithm is expressed as:

$$\vec{u} = \nabla \times \psi$$

Where: - $\vec{u}$ = resulting velocity vector - ∇ = vector differential operator (nabla) - ψ = potential field (3D Simplex Noise)

The right side is the **cross product** of the nabla operator and the potential field - mathematically known as **rot A** (rotation/curl).

Expanded Form

Computing the cross product with partial derivatives:

$$\vec{u} = \left(\frac{\partial \psi_3}{\partial y} - \frac{\partial \psi_2}{\partial z}, \frac{\partial \psi_1}{\partial z} - \frac{\partial \psi_3}{\partial x}, \frac{\partial \psi_2}{\partial x} - \frac{\partial \psi_1}{\partial y} \right)$$

Intuitive Understanding

The rotation (curl) operation can be understood as: “twisted partial derivative lookups in each direction, with terms subtracted from each other to create rotation.”

The implementation samples the noise field at slightly offset positions in each axis direction, then performs the cross product calculation.

Mass Conservation (Divergence-Free)

Those familiar with fluid dynamics might wonder about **mass conservation** - the principle that fluid flow must have zero divergence (what flows in must flow out):

$$\nabla \cdot \vec{u} = 0$$

The elegant answer: gradient noise inherently varies smoothly (like a 2D gradient where darker pixels on one side mean lighter on the other). This smooth variation naturally guarantees divergence-free behavior in the potential field. The mathematical structure ensures mass conservation without explicit enforcement.

GPU Implementation

The algorithm translates to remarkably simple shader code:

```
#define EPSILON 1e-3

float3 CurlNoise(float3 coord)
{
    float3 dx = float3(EPSILON, 0.0, 0.0);
    float3 dy = float3(0.0, EPSILON, 0.0);
    float3 dz = float3(0.0, 0.0, EPSILON);

    // Sample noise at offset positions
    float3 dpdx0 = snoise(coord - dx);
    float3 dpdx1 = snoise(coord + dx);
    float3 dpdy0 = snoise(coord - dy);
    float3 dpdy1 = snoise(coord + dy);
    float3 dpdz0 = snoise(coord - dz);
    float3 dpdz1 = snoise(coord + dz);

    // Cross product calculation
    float x = dpdy1.z - dpdy0.z + dpdz1.y - dpdz0.y;
    float y = dpdz1.x - dpdz0.x + dpdx1.z - dpdx0.z;
    float z = dpdx1.y - dpdx0.y + dpdy1.x - dpdy0.x;

    return float3(x, y, z) / EPSILON * 2.0;
}
```

Implementation Breakdown

Step	Description
Define EPSILON	Small offset for finite difference approximation
Sample noise at 6 positions	+/- offset in each axis direction
Compute cross product terms	Difference of perpendicular components
Scale result	Divide by epsilon and scale

Practical Applications

The sample Compute Shader implementation demonstrates various effects:

1. **Particle Advection** - Move particles along the curl noise velocity field
2. **Fire/Smoke Effects** - Add upward bias vectors to create flame-like appearance
3. **Abstract Fluid Art** - Creative applications limited only by imagination

Visual Examples

The repository includes examples showing: - Swirling particle systems - Flame-like upward flows - Organic, flowing motion patterns

---

Key Advantages

Aspect	Benefit
Computational Cost	Much lighter than full fluid simulation
Implementation	Simple arithmetic operations
Scalability	Runs well at high resolutions
Quality	Produces convincing fluid-like motion
Divergence-Free	Guaranteed by mathematical structure

Key Takeaways

1. **Curl Noise derives velocity from noise** - Uses gradient noise as potential field
2. **Cross product creates rotation** - The curl operation naturally produces swirling motion
3. **Inherently divergence-free** - No explicit mass conservation step needed
4. **Extremely efficient** - Just noise samples and arithmetic
5. **GPU-friendly** - Parallelizes perfectly in compute shaders
6. **High resolution capable** - Ideal for 4K/8K real-time rendering

---

Summary

Curl Noise enables 3D pseudo-fluid effects with minimal computational overhead. For high-resolution real-time rendering, it's an invaluable tool in the graphics programmer's arsenal.

This chapter concludes with gratitude to Professor Robert Bridson, who continues to develop innovative techniques like this algorithm.

References

- Robert Bridson, Jim Hourihan, Marcus Nordenstam. 2007. "Curl-noise for procedural fluid flow." In Proceedings of ACM SIGGRAPH 46.

Note: Curl Noise is a powerful starting point - combine it with additional forces, boundaries, and particle systems to create sophisticated fluid-like effects.

Chapter 7: Shape Matching - Applying Linear Algebra to CG

Author: Takao

Sample Project: "ShapeMatching" in the Unity Graphics Programming 2 Repository

---

Introduction

This chapter covers the fundamentals and applications of linear algebra, leading to Singular Value Decomposition (SVD) and its application in Shape Matching. Many people learn matrices in high school and linear algebra in university but struggle to see how these concepts apply in CG - this chapter bridges that gap.

For accessibility, explanations are limited to **2D with real numbers**. While some definitions differ slightly from formal linear algebra, the core concepts remain valid.

Part 1: Matrix Fundamentals Review

What is a Matrix?

A matrix is numbers arranged in rows and columns:

$$M = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Terminology: - **Row** - horizontal direction - **Column** - vertical direction - **Diagonal** - from top-left to bottom-right - **Elements** - individual numbers - **Matrix** - "Matrix" in English

Basic Operations

For 2x2 matrices **A** and **B**, and vector **c**:

Addition - element-wise:

$$A + B = \begin{pmatrix} a_{00} + b_{00} & a_{01} + b_{01} \\ a_{10} + b_{10} & a_{11} + b_{11} \end{pmatrix}$$

Subtraction - element-wise:

$$A - B = \begin{pmatrix} a_{00} - b_{00} & a_{01} - b_{01} \\ a_{10} - b_{10} & a_{11} - b_{11} \end{pmatrix}$$

Multiplication - more complex, order matters:

$$AB \neq BA$$

(in general)

Inverse Matrix - the "division" equivalent: - For scalars: $4 \times \frac{1}{4} = 1$ - For matrices: $M \cdot M^{-1} = M^{-1} \cdot M = I$

The **identity matrix** I is the matrix equivalent of 1:

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Inverse formula for 2x2:

$$A^{-1} = \frac{1}{a_{00}a_{11} - a_{01}a_{10}} \begin{pmatrix} a_{11} & -a_{01} \\ -a_{10} & a_{00} \end{pmatrix}$$

The denominator $(a_{00}a_{11} - a_{01}a_{10})$ is the **determinant**, written $\det(A)$.

Matrix-Vector Multiplication (Transformation)

Matrices transform vectors - the foundation of coordinate transformations in CG:

$$Ac = \begin{pmatrix} a_{00}c_0 + a_{01}c_1 \\ a_{10}c_0 + a_{11}c_1 \end{pmatrix}$$

Part 2: Advanced Matrix Concepts

Transpose

Swap rows and columns:

$$A^T = \begin{pmatrix} a_{00} & a_{10} \\ a_{01} & a_{11} \end{pmatrix}$$

Symmetric Matrix

A matrix where $A^T = A$.

Eigenvalues and Eigenvectors

For a square matrix **A**, if:

$$A\vec{v} = \lambda\vec{v}$$

(where $\vec{v} \neq 0$)

Then λ is an **eigenvalue** and $\vec{v}$ is an **eigenvector**.

Intuition: Eigenvectors are special directions where the matrix only scales (by λ), without rotation.

Computing eigenvalues: Solve $\det(A - \lambda I) = 0$, which yields a polynomial equation.

Eigenvalue Decomposition

A matrix can be decomposed as:

$$A = V\Lambda V^{-1}$$

Where: - Λ = diagonal matrix of eigenvalues (sorted) - V = columns are corresponding eigenvectors

Orthonormal Basis

Vectors that are: - Mutually perpendicular - All unit length (magnitude = 1)

Example: standard x and y axes: $(1, 0)$ and $(0, 1)$

Orthogonal Matrix

A matrix **Q** where columns form an orthonormal set:

$$Q^T Q = I, \quad Q^{-1} = Q^T$$

Part 3: Singular Value Decomposition (SVD)

Any m x n matrix **A** can be decomposed as:

$$A = U\Sigma V^T$$

Where: - **U** = m x m orthogonal matrix - Σ = m x n diagonal matrix (non-negative values, sorted) - **V^T** = n x n orthogonal matrix

Why SVD Matters

Unlike eigenvalue decomposition (only for square matrices), SVD works on **any** matrix.

Key Property

For a symmetric matrix, eigenvalues and singular values are identical.

Computing SVD

Multiply both sides by A^T:

$$A^T A = V\Sigma^T \Sigma V^T = V\Sigma^2 V^T$$

This matches eigenvalue decomposition form! So: 1. Compute eigenvalues of $A^T A$ 2. Square roots of eigenvalues = singular values 3. Eigenvectors of $A^T A$ = columns of V 4. Compute U from the relationship

C# Implementation

```
public void SVD(ref Matrix2x2 u, ref Matrix2x2 s, ref Matrix2x2 v)
{
    // Special case: diagonal matrix
    if (Mathf.Abs(this[1, 0] - this[0, 1]) < MATRIX_EPSILON
        && Mathf.Abs(this[1, 0]) < MATRIX_EPSILON)
    {
        u.SetValue(this[0, 0] < 0 ? -1 : 1, 0,
                   0, this[1, 1] < 0 ? -1 : 1);
        s.SetValue(Mathf.Abs(this[0, 0]), Mathf.Abs(this[1, 1]));
        v.LoadIdentity();
    }
    else
    {
        // Compute A^T * A terms
        float i = this[0, 0] * this[0, 0] + this[1, 0] * this[1, 0];
        float j = this[0, 1] * this[0, 1] + this[1, 1] * this[1, 1];
        float i_dot_j = this[0, 0] * this[0, 1] + this[1, 0] * this[1, 1];

        // Orthogonal case
```

```
        if (Mathf.Abs(i_dot_j) < MATRIX_EPSILON)
        {
            float s1 = Mathf.Sqrt(i);
            float s2 = Mathf.Abs(i - j) < MATRIX_EPSILON ? s1 : Mathf.Sqrt(j);
            u.SetValue(this[0, 0] / s1, this[0, 1] / s2,
                       this[1, 0] / s1, this[1, 1] / s2);
            s.SetValue(s1, s2);
            v.LoadIdentity();
        }
        // General case: solve quadratic for eigenvalues
        else
        {
            float i_minus_j = i - j;
            float i_plus_j = i + j;
            float root = Mathf.Sqrt(i_minus_j * i_minus_j + 4 * i_dot_j * i_dot_j);
            float eig = (i_plus_j + root) * 0.5f;
            float s1 = Mathf.Sqrt(eig);
            float s2 = Mathf.Abs(root) < MATRIX_EPSILON ? s1 :
                       Mathf.Sqrt((i_plus_j - root) / 2);

            s.SetValue(s1, s2);

            // V from eigenvectors
            float v_s = eig - i;
            float len = Mathf.Sqrt(v_s * v_s + i_dot_j * i_dot_j);
            i_dot_j /= len;
            v_s /= len;
            v.SetValue(i_dot_j, -v_s, v_s, i_dot_j);

            // U = A * V * S^-1
            u.SetValue(
                (this[0, 0] * i_dot_j + this[0, 1] * v_s) / s1,
                (this[0, 1] * i_dot_j - this[0, 0] * v_s) / s2,
                (this[1, 0] * i_dot_j + this[1, 1] * v_s) / s1,
                (this[1, 1] * i_dot_j - this[1, 0] * v_s) / s2
            );
        }
    }
}
```

Part 4: Shape Matching

Overview

Shape Matching aligns two different shapes with minimal error. It's used in: - Soft body physics simulation - Point cloud registration - Motion capture fitting

The Problem

Given two point sets P (target) and Q (source) with corresponding points, find rotation **R** and translation **t** to align Q onto P.

Algorithm

1. **Compute centroids** of both point sets:

$$\vec{p} = \frac{1}{n}\sum_{i=1}^{n}\vec{p}_i, \quad \vec{q} = \frac{1}{n}\sum_{i=1}^{n}\vec{q}_i$$

2. **Center both sets** (subtract centroids):

$$\vec{p}'_i = \vec{p}_i - \vec{p}, \quad \vec{q}'_i = \vec{q}_i - \vec{q}$$

3. **Compute covariance matrix H**:

$$H = \sum_{i=1}^{n}\vec{q}'_i(\vec{p}'_i)^\top$$

   Note: This is the **outer product** (creates a matrix from two vectors)

4. **SVD decomposition** of H:

$$H = U\Sigma V^T$$

5. **Extract rotation**:

$$R = VU^T$$

6. **Compute translation**:

$$\vec{t} = \vec{p} - R\vec{q}$$

Implementation

```
void Start()
{
    // Initialize point sets and centroids
    p = new Vector2[n];
    q = new Vector2[n];
    centerP = Vector2.zero;
    centerQ = Vector2.zero;

    // Gather points and compute centroids
    for(int i = 0; i < n; i++)
    {
        p[i] = _destination.transform.GetChild(i).position;
        centerP += p[i];
        q[i] = _target.transform.GetChild(i).position;
        centerQ += q[i];
    }
    centerP /= n;
    centerQ /= n;

    // Center points and build covariance matrix
    Matrix2x2 H = new Matrix2x2(0, 0, 0, 0);
    for (int i = 0; i < n; i++)
    {
        p[i] = p[i] - centerP;
        q[i] = q[i] - centerQ;
```

```
        H += Matrix2x2.OuterProduct(q[i], p[i]);
    }

    // SVD
    Matrix2x2 u = new Matrix2x2();
    Matrix2x2 s = new Matrix2x2();
    Matrix2x2 v = new Matrix2x2();
    H.SVD(ref u, ref s, ref v);

    // Extract rotation and translation
    R = v * u.Transpose();
    t = centerP - R * centerQ;
}
```

Key Takeaways

Concept	Application
SVD	Decomposes any matrix into rotation-scale-rotation
Covariance Matrix	Captures relationship between point sets
Shape Matching	Finds optimal rigid transformation between shapes
Outer Product	Builds matrices from vector pairs

Applications of SVD in CG

1. **Shape Matching** - Soft body simulation, registration
2. **Anisotropic Kernels** - Fluid surface reconstruction
3. **Material Point Method** - Snow/material simulation
4. **Principal Component Analysis** - Dimensionality reduction
5. **Image Compression** - Low-rank approximations

Summary

SVD provides a powerful tool for extracting rotation and scale from transformation data. Shape Matching demonstrates how linear algebra concepts translate into practical CG algorithms.

While this chapter covered 2D, the same algorithms extend directly to 3D - only the matrix sizes change.

References

- “3D Geometry for Computer Graphics” (ETH Zurich)
- MIT OpenCourseWare: “The Singular Value Decomposition (SVD)”
- AMATH 301: “The Singular Value Decomposition (SVD)” lecture
- Qiita: “Visualizing Eigenvalues and Eigenvectors” by @kenmatsu4

Chapter 8: Space Filling - Apollonian Gaskets

Author: Aoyama

Sample Project: “SpaceFilling” in the Unity Graphics Programming 2 Repository

---

Introduction

This chapter focuses on **Space Filling problems** and explores one solution: the **Apollonian Gasket**. While this chapter diverges slightly from typical graphics programming, the underlying algorithms have fascinating visual applications.

What is the Space Filling Problem?

The Space Filling problem asks: how do you fill a closed region with shapes as completely as possible, without overlapping?

This problem has been studied extensively in geometry and combinatorial optimization. Different shape combinations require different approaches:

Problem Type	Method
Rectangle Packing	O-Tree Method
Polygon Packing	Bottom-Left Method
Circle Packing	Apollonian Gasket
Triangle Packing	Sierpinski Gasket

Important Note: Space filling is NP-hard, meaning no algorithm guarantees 100% coverage. The Apollonian Gasket is no exception - it cannot completely fill a circular area with circles.

The Apollonian Gasket

Historical Background

The Apollonian Gasket is a type of fractal figure generated from three mutually tangent circles. Named after the ancient Greek mathematician Apollonius of Perga, it emerged from his work in planar geometry rather than as a space-filling algorithm.

The Core Concept

Given three mutually tangent circles C1, C2, C3, Apollonius discovered that exactly two non-intersecting circles (C4, C5) exist that are tangent to all three.

From here, consider (C1, C2, C4) - this trio has two tangent circles: C3 (already known) and a new circle C6. Repeat this process for all combinations: - (C1, C2, C5) - (C2, C3, C4) - (C1, C3, C4) - etc.

Repeating infinitely produces the Apollonian Gasket - an infinite set of mutually tangent circles.

Key Terminology

Apollonius Circle: Historically, the locus of points P where AP:BP = constant. In this context, it refers to solutions to Apollonius's Problem.

Apollonius's Problem: Given three circles, find a fourth circle tangent to all three. Up to 8 solutions exist, with 2 always externally tangent and 2 always internally tangent.

Implementation

Prerequisites: Helper Classes

Circle Class:

```
public class Circle
{
    public float Curvature => 1f / this.radius;
    public Complex Complex { get; private set; }
    public float Radius => Mathf.Abs(this.radius);
    public Vector2 Position => this.Complex.Vec2;

    private float radius = 0f;

    public Circle(Complex complex, float radius)
    {
        this.radius = radius;
        this.Complex = complex;
    }

    // Methods for checking relationships:
    // IsCircumscribed, IsInscribed, etc.
}
```

Complex Number Struct: Required for the complex form of Descartes' Circle Theorem (C# System.Numerics.Complex requires .NET 4.0+, so a custom implementation is provided).

Step 1: Creating Initial Three Circles

The algorithm requires three mutually tangent circles as input. These are generated randomly:

```
private void CreateFirstCircles(out Circle c1, out Circle c2, out Circle c3)
{
    var r1 = Random.Range(firstRadiusMin, firstRadiusMax);
    var r2 = Random.Range(firstRadiusMin, firstRadiusMax);
    var r3 = Random.Range(firstRadiusMin, firstRadiusMax);

    // First circle: random position
    var p1 = GetRandPosInCircle(fieldRadiusMin, fieldRadiusMax);
    c1 = new Circle(new Complex(p1), r1);

    // Second circle: tangent to first
    var p2 = -p1.normalized * ((r1 - p1.magnitude) + r2);
    c2 = new Circle(new Complex(p2), r2);
```

```
    // Third circle: tangent to both (using law of cosines)
    var p3 = GetThirdVertex(p1, p2, r1 + r2, r2 + r3, r1 + r3);
    c3 = new Circle(new Complex(p3), r3);
}
```

Law of Cosines Application: The three circle centers form a triangle where each side equals the sum of two radii. Using:

$$\cos\alpha = \frac{c^2 + b^2 - a^2}{2cb}$$

We can find the angle, then compute the third center's position.

Step 2: Descartes' Circle Theorem (Computing Radius)

For four mutually tangent circles with curvatures k1, k2, k3, k4:

$$(k_1 + k_2 + k_3 + k_4)^2 = 2(k_1^2 + k_2^2 + k_3^2 + k_4^2)$$

Where **curvature** k = 1/radius.

Solving for k4:

$$k_4 = k_1 + k_2 + k_3 \pm 2\sqrt{k_1k_2 + k_2k_3 + k_3k_1}$$

Two Solutions: - Positive curvature: circle externally tangent to all three - Negative curvature: circle internally tangent (contains all three)

```
var k1 = Circle1.Curvature;
var k2 = Circle2.Curvature;
var k3 = Circle3.Curvature;

var plusK = k1 + k2 + k3 + 2f * Mathf.Sqrt(k1 * k2 + k2 * k3 + k3 * k1);
var minusK = k1 + k2 + k3 - 2f * Mathf.Sqrt(k1 * k2 + k2 * k3 + k3 * k1);
```

Note: These four tangent circles are called **Soddy Circles** (after chemist Frederick Soddy who rediscovered the theorem).

Step 3: Descartes' Complex Circle Theorem (Computing Center)

For centers as complex numbers z1, z2, z3, z4:

$$(k_1z_1 + k_2z_2 + k_3z_3 + k_4z_4)^2 = 2(k_1^2z_1^2 + k_2^2z_2^2 + k_3^2z_3^2 + k_4^2z_4^2)$$

Solving for z4:

$$z_4 = \frac{z_1k_1 + z_2k_2 + z_3k_3 \pm 2\sqrt{k_1k_2z_1z_2 + k_2k_3z_2z_3 + k_3k_1z_3z_1}}{k_4}$$

```
var ck1 = Complex.Multiply(Circle1.Complex, k1);
var ck2 = Complex.Multiply(Circle2.Complex, k2);
var ck3 = Complex.Multiply(Circle3.Complex, k3);

var plusZ = ck1 + ck2 + ck3
    + Complex.Multiply(Complex.Sqrt(ck1 * ck2 + ck2 * ck3 + ck3 * ck1), 2f);
var minusZ = ck1 + ck2 + ck3
    - Complex.Multiply(Complex.Sqrt(ck1 * ck2 + ck2 * ck3 + ck3 * ck1), 2f);
```

Validation: Unlike radius, one of the two center solutions is correct. Test each candidate:

```
(c1.IsCircumscribed(c4, accuracy) || c1.IsInscribed(c4, accuracy)) &&
(c2.IsCircumscribed(c4, accuracy) || c2.IsInscribed(c4, accuracy)) &&
(c3.IsCircumscribed(c4, accuracy) || c3.IsInscribed(c4, accuracy))
```

Step 4: Recursive Generation

```
private void Awake()
{
    // Create initial three circles
    Circle c1, c2, c3;
    CreateFirstCircles(out c1, out c2, out c3);
    circles.Add(c1);
    circles.Add(c2);
    circles.Add(c3);

    soddys.Enqueue(new SoddyCircles(c1, c2, c3));

    while(soddys.Count > 0)
    {
        var soddy = soddys.Dequeue();
        Circle c4, c5;
        soddy.GetApollonianGaskets(out c4, out c5);
        AddCircle(c4, soddy);
        AddCircle(c5, soddy);
    }
}

private void AddCircle(Circle c, SoddyCircles soddy)
{
    if(c == null || c.Radius <= MinimumRadius) return;

    // Negative curvature circles appear only once
    if(c.Curvature < 0f)
    {
        circles.Add(c);
        soddy.GetSoddyCircles(c).ForEach(s => soddys.Enqueue(s));
        return;
    }

    // Check for overlaps with existing circles
    for(var i = 0; i < circles.Count; i++)
    {
```

```
            var o = circles[i];
            if(o.Curvature < 0f) continue;
            if(o.IsMatch(c, CalculationAccuracy)) return;
        }

        circles.Add(c);
        soddy.GetSoddyCircles(c).ForEach(s => soddys.Enqueue(s));
    }
```

Termination Condition: Since infinite recursion is impossible, stop when new circles become smaller than `MinimumRadius`.

Overlap Handling: New circles might overlap with existing ones even if geometrically valid - these must be filtered out.

Key Takeaways

Concept	Description
Curvature	1/radius - enables elegant mathematical formulations
Descartes' Theorem	Relates curvatures of four mutually tangent circles
Complex Plane	Natural representation for circle centers in 2D
Soddy Circles	Four mutually tangent circles satisfying Descartes' theorem
Fractal Nature	Infinite recursion produces self-similar patterns

---

Extensions and Applications

3D Space Filling

Moving from circles to spheres dramatically increases complexity. The famous **Kepler Conjecture** about sphere packing took centuries to prove mathematically.

Practical Applications

- **VLSI Layout** - Chip design optimization
- **Material Cutting** - Minimizing waste in fabric/material cutting
- **UV Unwrapping** - Automatic optimization of texture space
- **Procedural Art** - Generative visual patterns

Summary

The Apollonian Gasket demonstrates how classical geometry can address modern computational problems. While historically a fractal curiosity, the underlying algorithms for fitting circles within circles have practical applications in layout optimization and generative art.

The concept of filling space with shapes opens creative possibilities for visual expression in unexpected ways.

References

- Wikipedia: Apollonian Gasket
- Wikipedia: Descartes' Circle Theorem
- Paul Bourke: Random Tiling

Chapter 9: Image Effect Introduction

Author: (SimpleImageEffect)

Sample Project: "SimpleImageEffect" in the Unity Graphics Programming 2 Repository

---

Introduction

This chapter provides a simple explanation of how to implement **Image Effects** (also called **Post Effects**) in Unity - the technique of applying GPU shaders to the rendered output.

Image Effects are used for: - Glow/bloom effects - Anti-aliasing - Depth of Field (DOF) - Color correction and grading - Many other visual enhancements

The simplest examples involve color modifications, which this chapter covers.

Prerequisites: Basic familiarity with Unlit or Surface shaders is helpful, but the simple structure makes this accessible even without prior shader experience.

How Image Effects Work

Image Effects process the rendered image pixel-by-pixel. Essentially, implementing an Image Effect means implementing a **fragment shader** that operates on the screen output.

Unity's Image Effect Pipeline

1. Camera renders the scene
2. Rendered content is passed to `OnRenderImage` method
3. Image Effect shader processes the input
4. `OnRenderImage` outputs the modified image

Basic Implementation

The C# Script

Scene: "ImageEffectBase"

```
[ExecuteInEditMode]
[RequireComponent(typeof(Camera))]
public class ImageEffectBase : MonoBehaviour
{
    public Material material;
```

```
    protected virtual void Start()
    {
        if (!SystemInfo.supportsImageEffects
         || !this.material
         || !this.material.shader.isSupported)
        {
            base.enabled = false;
        }
    }

    protected virtual void OnRenderImage(RenderTexture source, RenderTexture destination)
    {
        Graphics.Blit(source, destination, this.material);
    }
}
```

Key Components

Attributes: - `[ExecuteInEditMode]` - See effect in Scene view without playing - `[RequireComponent(typeof(Camera))]` - `OnRenderImage` only works with cameras

OnRenderImage Parameters: - `source` - The camera's rendered output - `destination` - Where to write the result (null = screen buffer)

Graphics.Blit: - Renders `source` to `destination` using the specified material - Must always write something to `destination`

Validation Note

The `Start()` validation disables the effect on unsupported hardware. Be aware that `SystemInfo.supportsImageEffects` checks might fail in `Awake` or `OnEnable` due to `ExecuteInEditMode` timing.

The Simplest Image Effect Shader

```
Shader "ImageEffectBase"
{
    Properties
    {
        _MainTex("Texture", 2D) = "white" {}
    }
    SubShader
    {
        Cull Off ZWrite Off ZTest Always

        Pass
        {
            CGPROGRAM

            #include "UnityCG.cginc"
            #pragma vertex vert_img
            #pragma fragment frag
```

```
            sampler2D _MainTex;

            fixed4 frag(v2f_img input) : SV_Target
            {
                float4 color = tex2D(_MainTex, input.uv);
                color.rgb = 1 - color.rgb;  // Invert colors
                return color;
            }

            ENDCG
        }
    }
}
```

Understanding the Shader

`_MainTex`: Reserved name for `Graphics.Blit` input. Changing this name breaks the pipeline.

`Cull Off ZWrite Off ZTest Always`: Unity recommends these settings to prevent unintended depth buffer interactions.

`#pragma vertex vert_img`: Uses Unity's built-in vertex shader from UnityCG.cginc - handles the full-screen quad rendering.

`v2f_img`: Built-in structure containing position and UV coordinates.

Why the Simplified Version Works

Unity's default Image Effect shader includes a complete vertex shader:

```
struct appdata
{
    float4 vertex : POSITION;
    float2 uv : TEXCOORD0;
};

struct v2f
{
    float2 uv : TEXCOORD0;
    float4 vertex : SV_POSITION;
};

v2f vert (appdata v)
{
    v2f o;
    o.vertex = UnityObjectToClipPos(v.vertex);
    o.uv = v.uv;
    return o;
}
```

The vertex shader just creates a screen-aligned quad with UV coordinates - rarely needs modification. UnityCG.cginc provides `vert_img` and `v2f_img` for this common case.

Practice: Diagonal Split Effect

Let's modify the effect to only invert the diagonal half of the screen.

Understanding UV Coordinates

`input.uv` contains normalized (0-1) coordinates where: - Origin (0,0) = bottom-left - (1,1) = top-right

Horizontal/Vertical Split

```
// Invert left half
color.rgb = input.uv.x < 0.5 ? 1 - color.rgb : color.rgb;

// Invert bottom half
color.rgb = input.uv.y < 0.5 ? 1 - color.rgb : color.rgb;
```

Diagonal Split

```
color.rgb = input.uv.y < input.uv.x ? 1 - color.rgb : color.rgb;
```

Useful Built-in Values

_ScreenParams

`float4` containing: - x = pixel width - y = pixel height - z = 1 + 1/width - w = 1 + 1/height

Useful for aspect ratio calculations and pixel-based effects.

_MainTex_TexelSize

`float4` containing: - x = 1/width - y = 1/height - z = width - w = height

Automatically defined for any `sampler2D` - just add `_TexelSize` suffix.

Sampling Adjacent Pixels

```
sampler2D _MainTex;
float4    _MainTex_TexelSize;

fixed4 frag(v2f_img input) : SV_Target
{
    float4 color = tex2D(_MainTex, input.uv);

    // Sample 4 neighbors
    color += tex2D(_MainTex, input.uv + float2(_MainTex_TexelSize.x, 0));
    color += tex2D(_MainTex, input.uv - float2(_MainTex_TexelSize.x, 0));
    color += tex2D(_MainTex, input.uv + float2(0, _MainTex_TexelSize.y));
    color += tex2D(_MainTex, input.uv - float2(0, _MainTex_TexelSize.y));

    return color / 5;  // Average (smoothing filter)
}
```

This pattern is used for blur filters, edge detection, and many other effects.

Accessing Depth and Normals

G-Buffer Concept

The G-Buffer stores per-pixel information beyond color: - Depth buffer - Normal buffer - Other material properties

This data enables effects like ambient occlusion, screen-space reflections, and more.

Enabling Depth/Normal Access

```
public class ImageEffect : ImageEffectBase
{
    protected new Camera camera;
    public DepthTextureMode depthTextureMode;

    protected override void Start()
    {
        base.Start();
        this.camera = base.GetComponent<Camera>();
        this.camera.depthTextureMode = this.depthTextureMode;
    }
}
```

DepthTextureMode Options: - None (default) - `Depth` - depth only - `DepthNormals` - depth and normals combined - `MotionVectors` - per-pixel motion

Reading Depth and Normals in Shader

```
sampler2D _MainTex;
sampler2D _CameraDepthNormalsTexture;

fixed4 frag(v2f_img input) : SV_Target
{
    float4 color = tex2D(_MainTex, input.uv);
    float3 normal;
    float  depth;

    // Decode combined depth+normal texture
    DecodeDepthNormal(
        tex2D(_CameraDepthNormalsTexture, input.uv),
        depth,
        normal
    );

    // Normalize depth to 0-1 range (platform-independent)
    depth = Linear01Depth(depth);

    // Visualize depth
    return fixed4(depth, depth, depth, 1);

    // Or visualize normals
    return fixed4(normal.xyz, 1);
}
```

Depth Visualization

- Near objects: depth closer to 1 (white)
- Far objects: depth closer to 0 (black)
- Affected by camera's near/far clip planes

Normal Visualization

Normal components (X, Y, Z) map to (R, G, B): - Surfaces facing right: more red - Surfaces facing up: more green - Surfaces facing camera: more blue

---

Key Takeaways

Concept	Description
OnRenderImage	Unity callback for post-processing
Graphics.Blit	Renders texture through material to destination
_MainTex	Reserved name for Blit input
vert_img	Built-in vertex shader for full-screen effects
_TexelSize	Automatic pixel size for any sampler2D
DepthTextureMode	Enables G-buffer access
DecodeDepthNormal	Extracts depth and normal from combined texture

---

Summary

Image Effects transform rendered output through fragment shaders. The pipeline is straightforward:

1. Camera renders to texture
2. `OnRenderImage` receives the texture
3. `Graphics.Blit` processes it through your shader
4. Result goes to screen (or next effect)

With depth and normal access, you can create sophisticated effects like ambient occlusion, fog, outlines, and more - topics explored in the next chapter.

References

- Writing Image Effects - Unity Documentation
- Accessing shader properties in Cg/HLSL
- Using Depth Textures

Chapter 10: Image Effect Application - Screen Space Reflection (SSR)

Author: Komietty

Sample Project: "SSR" in the Unity Graphics Programming 2 Repository

---

Introduction

This chapter covers **Screen Space Reflection (SSR)** as an advanced application of Image Effects. Reflections and specular highlights are crucial for creating realistic 3D environments, yet physically accurate ray tracing requires enormous computational resources.

While Unity now supports tools like Octane Renderer for offline rendering, real-time applications still require clever approximations. SSR is a post-processing technique that achieves convincing reflections within the constraints of screen-space data.

Chapter Structure

1. **Gaussian Blur** - Warm-up post-effect technique
2. **SSR Core Algorithm** - Ray marching in screen space
3. **Optimization Techniques** - Mipmap acceleration, binary search refinement

---

Part 1: Gaussian Blur

The Concept

Blur effects work by sampling neighboring pixels and blending their colors. The **kernel** determines the blending weights. Gaussian blur uses a normal distribution for natural-looking results.

Gaussian Distribution

$$G(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{x^2}{2\sigma^2}\right)$$

In practice, we approximate this with binomial distribution weights:

```
float4 x_blur (v2f i) : SV_Target
{
    float weight[5] = { 0.2270270, 0.1945945, 0.1216216, 0.0540540, 0.0162162 };
    float offset[5] = { 0.0, 1.0, 2.0, 3.0, 4.0 };
    float2 size = _MainTex_TexelSize;

    fixed4 col = tex2D(_MainTex, i.uv) * weight[0];
    for(int j = 1; j < 5; j++)
    {
        col += tex2D(_MainTex, i.uv + float2(offset[j], 0) * size) * weight[j];
        col += tex2D(_MainTex, i.uv - float2(offset[j], 0) * size) * weight[j];
    }
    return col;
}
```

Separable Blur Optimization

By splitting into X and Y passes, we reduce texture samples from n x n to 2n + 1:

```
void OnRenderImage(RenderTexture src, RenderTexture dst)
{
    var rt = RenderTexture.GetTemporary(src.width, src.height, 0, src.format);

    for (int i = 0; i < blurNum; i++)
```

```
    {
        Graphics.Blit(src, rt, mat, 0);  // X blur
        Graphics.Blit(rt, src, mat, 1);  // Y blur
    }
    Graphics.Blit(src, dst);

    RenderTexture.ReleaseTemporary(rt);
}
```

Part 2: SSR Fundamentals

Required Buffers

SSR needs: - **Color buffer** - The rendered image - **Depth buffer** - Per-pixel depth values - **Normal buffer** - Per-pixel surface normals

These G-buffers are essential for deferred rendering techniques.

The Core Idea

Unlike physical optics where light travels from source to eye, SSR works **backwards**: 1. For each pixel, determine the reflection direction 2. March a ray along that direction 3. Find where the ray hits geometry (using depth comparison) 4. Sample the color at that hit point

Algorithm Overview

1. Convert screen coordinates to world space using depth
2. Calculate reflection vector from view direction and normal
3. Step the ray forward in small increments
4. Transform ray position back to screen space
5. Compare ray depth with depth buffer
6. If ray is behind geometry, we found a reflection source
7. Sample and apply that color to the original pixel

Part 3: Implementation

Coordinate Transformation Setup

```
void OnRenderImage(RenderTexture src, RenderTexture dst)
{
    var view = cam.worldToCameraMatrix;
    var proj = GL.GetGPUProjectionMatrix(cam.projectionMatrix, false);
    var viewProj = proj * view;

    mat.SetMatrix("_ViewProj", viewProj);
    mat.SetMatrix("_InvViewProj", viewProj.inverse);
    // ...
}
```

Computing Reflection Direction

```
float4 reflection (v2f i) : SV_Target
{
    float2 uv = i.screen.xy / i.screen.w;
    float depth = SAMPLE_DEPTH_TEXTURE(_CameraDepthTexture, uv);

    // Reconstruct world position
    float2 screenpos = 2.0 * uv - 1.0;
    float4 pos = mul(_InvViewProj, float4(screenpos, depth, 1.0));
    pos /= pos.w;

    // Calculate view and reflection directions
    float3 camDir = normalize(pos - _WorldSpaceCameraPos);
    float3 normal = tex2D(_CameraGBufferTexture2, uv) * 2.0 - 1.0;
    float3 refDir = reflect(camDir, normal);

    // Debug visualization
    if (_ViewMode == 1) return float4((normal.xyz * 0.5 + 0.5), 1);
    if (_ViewMode == 2) return float4((refDir.xyz * 0.5 + 0.5), 1);
    // ...
}
```

Ray Marching Loop

```
[loop]
for (int n = 1; n <= _MaxLoop; n++)
{
    float3 step = refDir * _RayLenCoeff * (lod + 1);
    ray += step * (1 + rand(uv + _Time.x) * (1 - smooth));

    // Transform ray to screen space
    float4 rayScreen = mul(_ViewProj, float4(ray, 1.0));
    float2 rayUV = rayScreen.xy / rayScreen.w * 0.5 + 0.5;
    float rayDepth = ComputeDepth(rayScreen);

    // Sample world depth at ray position
    float worldDepth = (lod == 0) ?
        SAMPLE_DEPTH_TEXTURE(_CameraDepthTexture, rayUV) :
        tex2Dlod(_CameraDepthMipmap, float4(rayUV, 0, lod)) + _BaseRaise * lod;

    // Check for intersection
    if(rayDepth < worldDepth)
    {
        // Ray has passed behind geometry - found reflection!
        // ...
        return outcol;
    }
}
```

Note: The `[loop]` attribute is required when loop count is determined at runtime.

Part 4: Optimization Techniques

Challenge 1: Limited Ray Distance

With fixed loop count, rays can't travel far enough for distant reflections.

Challenge 2: Stepping Through Objects

Large steps may skip thin objects, sampling incorrect colors.

Challenge 3: Performance

Many iterations per pixel is expensive.

Solution: Hierarchical Ray Marching with Mipmaps

Use **mipmaps** (lower-resolution versions of the depth buffer) to take larger steps in empty space, then refine with smaller steps near geometry.

Creating Mipmapped RenderTexture

```
void OnEnable()
{
    rt = new RenderTexture(Screen.width, Screen.height, 24);
    rt.useMipMap = true;  // Enable mipmaps
}
```

LOD-Based Stepping

```
for (int n = 1; n <= _MaxLoop; n++)
{
    float3 step = refDir * _RayLenCoeff * (lod + 1);  // Larger steps at higher LOD
    ray += step;

    if(rayDepth < worldDepth)  // Intersection detected
    {
        if(lod == 0)
        {
            // Final refinement at full resolution
            // Binary search for precise hit point
            break;
        }
        else
        {
            ray -= step;  // Back up
            lod--;        // Decrease LOD for finer stepping
        }
    }
    else if(n <= _MaxLOD)
    {
        lod++;  // Increase LOD for larger steps in empty space
    }
}
```

Binary Search Refinement

Once we detect intersection at LOD 0, use binary search to find the precise hit point:

```
if (lod == 0)
{
    if (rayDepth + _Thickness > worldDepth)
    {
        float sign = -1.0;
        for (int m = 1; m <= 8; ++m)
        {
            ray += sign * pow(0.5, m) * step;

            rayScreen = mul(_ViewProj, float4(ray, 1.0));
            rayUV = rayScreen.xy / rayScreen.w * 0.5 + 0.5;
            rayDepth = ComputeDepth(rayScreen);
            worldDepth = SAMPLE_DEPTH_TEXTURE(_CameraDepthTexture, rayUV);

            sign = (rayDepth < worldDepth) ? -1 : 1;
        }
        refcol = tex2D(_MainTex, rayUV);
    }
    break;
}
```

Part 5: Material-Based Reflection

Using Smoothness from G-Buffer

Different materials should reflect differently. The G-buffer stores material properties:

```
// Sample material smoothness
float smooth = tex2D(_CameraGBufferTexture1, uv).w;

// Blend reflection based on smoothness
return (col * (1 - smooth) + refcol * smooth) * _ReflectionRate
     + col * (1 - _ReflectionRate);
```

Part 6: Edge-Aware Blur

Apply blur to hide artifacts from step size limitations, but preserve edges:

```
float4 xblur(v2f i) : SV_Target
{
    float2 uv = i.screen.xy / i.screen.w;
    float smooth = tex2D(_CameraGBufferTexture1, uv).w;

    // Edge detection via depth difference
    float depth = SAMPLE_DEPTH_TEXTURE(_CameraDepthTexture, uv);
    float depthR = SAMPLE_DEPTH_TEXTURE(_CameraDepthTexture, uv + float2(1, 0) * size);
    float depthL = SAMPLE_DEPTH_TEXTURE(_CameraDepthTexture, uv - float2(1, 0) * size);

    float4 originalColor = tex2D(_ReflectionTexture, uv);
```

```
    float4 blurredColor = /* Gaussian blur calculation */;

    // Skip blur at edges (large depth discontinuity)
    float4 o = (abs(depthR - depthL) > _BlurThreshold) ? originalColor
             : blurredColor * smooth + originalColor * (1 - smooth);
    return o;
}
```

Bonus: Two-Camera Reflections

An experimental technique using a secondary camera to reflect objects not visible to the main camera:

```
// Use sub-camera's depth and color buffers
float subCameraDepth = SAMPLE_DEPTH_TEXTURE(_SubCameraDepthTex, rayUv);

if (rayDepth < subCameraDepth && rayDepth + thickness > subCameraDepth)
{
    // Binary search refinement...
    col = tex2D(_SubCameraMainTex, rayUv);
}
```

Limitation: Objects that should be occluded may still appear in reflections.

Key Takeaways

Technique	Purpose
Ray Marching	Core SSR algorithm - step along reflection vector
Depth Comparison	Detect where rays intersect geometry
Hierarchical Tracing	Mipmap-based acceleration for longer rays
Binary Search	Precise intersection point refinement
Material Smoothness	Vary reflection intensity per-material
Edge-Aware Blur	Hide artifacts while preserving detail

Performance Considerations

SSR is computationally expensive because: - Processing scales with screen resolution - Each pixel may require many ray steps - Multiple texture samples per step

Optimization strategies: - Limit maximum ray steps - Use hierarchical depth (mipmaps) - Reduce reflection resolution - Blur to hide low sample counts

---

Summary

SSR achieves real-time reflections within screen-space constraints. While it cannot reflect objects outside the view frustum, it provides convincing results for floors, water, and glossy surfaces.

The techniques covered - ray marching, hierarchical acceleration, binary search refinement - extend beyond SSR to other screen-space effects like ambient occlusion and global illumination approximations.

Experiment with the sample project parameters to understand the trade-offs between quality and performance.

References

- Kode80: “Screen Space Reflections in Unity 5”
- Chalmers University: “Screen-space reflections” course materials
- hecomi: “Unity Screen Space Reflection Implementation”
- Unity Manual: Deferred Shading G-Buffer layout